

Protection Against Subversion Corruptions via Reverse Firewalls in the Plain Universal Composability Framework^{*}

Paula Arnold¹, Sebastian Berndt², Jörn Müller-Quade^{3,4}, and Astrid Ottenhues^{3,4}

¹ University of Luebeck

`p.arnold@uni-luebeck.de`

² Technische Hochschule Lübeck

`sebastian.berndt@th-luebeck.de`

³ Karlsruhe Institute of Technology (KIT), Karlsruhe, Germany

⁴ KASTEL Security Research Labs, Karlsruhe, Germany
`{mueller-quade, astrid.ottenhues}@kit.edu`

Abstract. While many modern cryptographic primitives have stood the test of time, attackers started to expand beyond classic cryptanalysis by targeting implementations. Subversion attacks, where the attacker replaces the implementation of the cryptographic primitive to leak sensitive information about the user during a protocol execution, are among the most dangerous of such attacks. The revelations of Snowden have shown that these attacks are deployed by intelligence services. A very promising countermeasure uses *cryptographic reverse firewalls* that actively remove the covert channel leaking the secret. Chakraborty et al. (EUROCRYPT’22) presented the first model of such firewalls in the universal composability (UC) framework. However, using such a firewall also provides a possible new target for the attacker and in the case that an honest party uses a corrupted firewall, they were not able to prove any security guarantees. Furthermore, their model is quite complex and does not fit into the *plain* UC model as they restrict the environment. Hence, the authors needed to reprove fundamental theorems such as the composition theorem as well as the security of the underlying protocol. In this paper, we consider a slightly different model of subversion attacks that replace the used randomness, inspired by Dodis et al. (CRYPTO’16), that captures all known subversion attacks. Considering these realistic attacks allows us to use *existing* UC-secure protocols without the need to reprove their security. We also introduce additional notions of firewall properties, allowing us to reason about corrupted firewalls while maintaining strong security guarantees. To show the versatility of our model, we apply it to commitments and oblivious transfer. This demonstrates the usefulness of our plain UC model, as the only known previous subversion-resilient OT, recently provided by Chakraborty et al. (ASIACRYPT’24), is much more complicated and involved, and was also in the non-plain UC model.

Keywords: Subversion Resilience · Universal Composability · OT

^{*} This is the full version of a paper published at ACNS 2025 [4].

Table of Contents

1	Introduction	3
1.1	Contributions	4
1.2	Technical Overview	5
1.3	Related Work	11
2	Preliminaries	12
2.1	Cryptographic Primitives	13
2.2	Universal Composability	13
3	Subversion Attacks and Countermeasures in UC	15
3.1	Modelling Subversion Corruptions in UC	16
3.2	Preventing Subversion Attacks in UC Using Reverse Firewalls	19
3.3	More Details on Corruptions Considered in This Paper	24
4	Properties of Reverse Firewalls	24
4.1	Fundamental Properties of Reverse Firewalls	25
4.2	Modelling Properties of Reverse Firewalls in UC	26
5	Security Analysis of Subversion Corrupted Protocols with Reverse Firewalls	27
5.1	How to Analyse Subversion Resilience of a Protocol	28
5.2	On Corruption Handling	31
5.3	Steps for Applying Our Model	34
6	Commitments	35
6.1	Ideal Functionality: Commitment	35
6.2	Real Protocol: Subversion-Resilient Commitment Based on DLog [21]	36
6.3	Security Proof	37
7	Oblivious Transfer	39
7.1	Ideal Functionality: Oblivious Transfer	39
7.2	Security Proof	41
8	Circumvention of an Impersonation Attack	43
8.1	On the Corruption Transition Table 2	44
8.2	Impersonation Attacks	44
8.3	Adding $\mathcal{F}_{\text{chAUTH}}$ to Π_{srCOM}	46
8.4	Circumventing Impersonation on Protocol Side	48
8.5	Remaining Security Guarantees in the Presence of an Impersonation	49
9	Conclusion	50
A	Comparison to Chakraborty et al.	54
A.1	Transparency	55
A.2	Corrupted Firewalls	57

1 Introduction

Nowadays, many cryptographic primitives such as AES or ECDH have stood the test of time and have been in use for decades without any major security loss due to improved attacks via cryptanalysis. However, already in the late seventies, Simmons noticed the danger of cryptographic implementations embedding a *covert channel* that can leak sensitive information only visible to the attacker [52]. Such attacks were later studied more formally under the name of *kleptography* by Young and Yung [54, 55]. While the existence of such attacks was thus clearly demonstrated on a theoretical level, no real-world attack was known so far. This drastically changed when the issues surrounding the Dual Elliptic Curve Deterministic Random Bit Generator (Dual_EC_DRBG) came to light that incorporated a backdoor provided by the NSA [27]. Furthermore, later revelations due to Snowden exposed the existence of NSA’s *Project Bullrun*. Some goals of this project were to “*Insert vulnerabilities into commercial encryption systems, IT systems, networks, and endpoint communications devices used by targets*” and to “*Influence policies, standards and specification for commercial public key technologies*” [47]. These revelations led to an increased alertness by cryptographic researchers that quickly started to develop possible countermeasures against such *subversion attacks* [8, 7]. While these countermeasures have not yet found their way into real-world systems, the danger of subversion attacks is actively discussed in real-world scenarios. For example, a specific countermeasure⁵ was proposed in the standardisation process of post-quantum cryptography to prevent such attacks [32], but was ultimately rejected.

Unfortunately, there are subversion attacks that are *completely undetectable* in a black-box setting [12] and all countermeasures thus need to use a non-black-box approach. One of the most widely studied countermeasures is the so-called *reverse firewall* [46, 33] that aims to actively remove the covert channel from a subverted implementation, often via means of rerandomisation. In general, such firewalls are devices placed between the different parties of a protocol, e.g., by being directly incorporated into routers. A cautious user wanting to protect their data against subverted implementations thus might decide to install such a firewall. While the use of a suitably chosen firewall can prevent subversion attacks, they can also enlarge the attack surface, as the firewall itself is now a new corruption target. Clearly, a cautious user would only use such a firewall if the additional attack surface is limited, i.e., if the firewall cannot lower the security guarantees significantly.

While different protocols and corresponding reverse firewalls have been presented in the literature, Chakraborty et al. [26] noticed that their use of game-based security notions could lead to problems when considering the security of concurrent runs of the underlying protocol. Hence, realistic attackers could still leak sensitive information via the covert channel by starting multiple concurrent sessions with the subverted protocol, even in the presence of these sophisticated countermeasures. To remedy this situation, they proposed an extension of the

⁵ This countermeasure has also been called the *hash of shame*.

Universal Composability (UC) model by Canetti [18, 16] to capture subversion attacks and countermeasures built on reverse firewalls. Based on this, they constructed a secure protocol for commitments and a reverse firewall for this protocol as well as a completeness theorem for maliciously secure MPC in their model by instantiating the Goldreich-Micali-Wigderson (GMW) compiler [36]. While these results are the first results guaranteeing security against subversion attacks in the context of concurrent runs, they come with a drawback. The original reverse firewalls guaranteed a high degree of *transparency*, i.e., neither the sender nor the receiver needed to be aware of the existence of the reverse firewall [46, 33]. Transparency allows for an elegant argument if the firewall itself is corrupted but the protocol is secure against malicious attackers: For each individual party, we can treat the corrupted firewall to be “external”, i.e., to be part of a different, maliciously corrupted party. Hence, the security of the original protocol directly implies security against a corrupted firewall. In contrast, in the model presented by Chakraborty et al., the sender is actively communicating with the reverse firewall via a *feedback channel* and thus needs to be aware of the existence of the firewall. Hence, the firewall cannot be transparent for the sender. At first glance, this looks like a minor annoyance without much effect. However, this has far-reaching consequences for the guarantees achievable by their model, as a corrupted firewall can now drastically reduce the security guarantees. Consider as an example that an honest, non-subverted implementation is used. In the model of Chakraborty et al., albeit giving this corruption case a different name, a maliciously corrupted firewall leads to the effect that the complete *combined party* (i.e., the party together with the firewall) will be treated as maliciously corrupted. But, as the implementation is honest, the firewall should not be able to extract any sensitive information from interacting with an honest party. Hence, there is a clear gap between the security guarantees provided by the model of Chakraborty et al. and the guarantees achieved in the real world. Note that the attacker model of Chakraborty et al. allows for very strong attacks, including attacks where the corrupted implementation completely deviates from its usual behaviour, e.g., by changing the message of a message transmission protocol. However, as discussed by Dodis et al. [33], protecting message transmission protocols against such functionality-changing attacks is very hard (or impossible, depending on the model). Hence, Chakraborty et al. need to focus on functionalities with *random* inputs or trust the firewall to not change the input. Furthermore, due to the complex generalisation of the already complex UC model, Chakraborty et al. need to reprove several fundamental results such as the *composition theorem*. Finally, as the authors use a generalisation of the UC model, to convert known UC-secure protocols to their model they need to essentially *reprove* the security of the protocol.

1.1 Contributions

In this paper, we present an alternative model to the one of Chakraborty et al. that is in plain UC which allows us to precisely capture and minimise the attack vectors available to a corrupted firewall. This alternative model follows

the threat model of Dodis et al. [33] by only considering corruptions preserving the underlying functionality. This allows us to also protect functionalities with non-random inputs, to reuse existing security proofs, and guarantee security in the presence of corrupted firewalls. We show the versatility of our approach by presenting UC-secure subversion resilient protocols for commitments and OT. In more detail, our contributions are:

- We present the first subversion corruption model in plain UC which allows us to use all of the existing tools such as the composition theorem and to also use existing UC-secure primitives.
- We show that it is possible to avoid subversion attacks already in real world protocols (i.e., $\Pi \geq_{\text{UC}} \Pi'$), such that there is no need to have the subversion corruption in the ideal world, while still having the security properties of subversion resilience (i.e., $\Pi \geq_{\text{UC}} \mathcal{F}$).
- To simplify the steps necessary when proving the UC-security of a protocol in the presence of subversion, we formalise the natural properties of a reverse firewall. This allows us to equip existing protocols with reverse firewalls and simplifies the security analysis significantly. We prove that one only needs to show that the firewalls guarantee *correctness*, *strong transparency*, and an *anti-signalling* notion to obtain a subversion-resilient protocol. Hence, we do not need to reprove the UC-security of the underlying protocol and can use existing, efficient protocols. Informally, we prove the following theorem.

Main Theorem (informal). *Given a protocol base Π' , we add a strongly transparent and anti-signalling reverse firewall to it to obtain protocol Π . Then, Π UC-emulates the protocol Π' . Therefore, the protocol Π is subversion-resilient and party-wise secure under the corruptions of Π' .*

- We show how to use our model to secure commitments against subversion attacks using an easy-to-follow guide.
- We use our guide to present a simple UC-secure subversion-resilient OT. In contrast to the only previous subversion-resilient OT of Chakraborty et al. [25], our OT is much simpler, more efficient and also works in the plain UC model instead of the non-plain UC model of [26].
- We precisely describe the attack possibilities of a corrupted firewall as impersonating the party. This allows us to give a more fine-granular security analysis avoiding the behaviour used in existing approaches, where the combined party was always treated as maliciously corrupted.
- We present several approaches to circumvent the impersonation attack vector of a corrupted firewall. Some of these are tailored to the specific protocol and thus highly efficient as adding unique signatures to the commitment scheme. Others are more expensive, but also general, as adding zero-knowledge proofs to, e.g., the OT scheme.

1.2 Technical Overview

In this section, we give a technical overview of the results of this paper: Our model for subversion in plain UC; Our model for reverse firewalls in plain UC;

How to handle malicious firewalls; How to prove security in the presence of subversion; Subversion-resilient commitments and oblivious transfer. For an illustrative example for readers not familiar with subversion attacks, we will use a running example to explain the used concepts.

Modelling Subversion (Section 3) Our first contribution is to model subversion attacks as *subversion corruptions* in the plain UC model, allowing to reuse a wide range of established results such as the composition theorem. Informally, to subvert a party $\mathcal{P}$ that runs some code π , the attacker $\mathcal{A}$, called subverter, replaces π by a *subverted code* $\bar{\pi}$ by issuing a corresponding subversion command that allows for temporal access to $\mathcal{P}$. After the replacement, this access is disabled and the party strictly follows the (new) code during the remaining protocol run.

Throughout this section, we will use the running example of subverting a UC-secure commitment protocol Com .

In general, the party will employ some software implementation of Com . A subverted implementation $\bar{\text{Com}}$ will now be provided by the subverter. After the implementation was obtained by the party, it will run $\bar{\text{Com}}(v, r)$ on its input v and a uniformly sampled randomness r and output the result c .

A party running $\bar{\pi}$ might behave differently than when running π . However, as the goal of the attacker is to avoid detection, this deviation needs to be limited. For example, since detecting this changed behaviour can lead to an abortion of the protocol by the honest parties or other countermeasures, it (and thus the subversion) should not be noticeable to the honest parties. Hence, we assume the attacker to use an *undetectable subversion* such that the behaviour of $\mathcal{P}$ running π is indistinguishable from the behaviour of $\mathcal{P}$ running $\bar{\pi}$ for other honest parties.

Usually, a subverter is described as wanting to break specific security guarantees of the underlying protocol, but we aim for a more general setting. We thus assume that $\mathcal{P}$ contains some *secret* sec that the subverter wants to obtain. While this can be, for example, cryptographic key material used to break the security guarantees, it can also be other sensitive information which was stored by other protocols on the device used by the party. To formalise this notion, we say that the subverted code $\bar{\pi}$ is *signalling* when the subverted code can send (or *signal*) some information to the attacker. In other words, $\mathcal{A}$ must be able to distinguish between the behaviour of $\mathcal{P}$ running π and $\mathcal{P}$ running $\bar{\pi}$.

Undetectable Subversions (Section 3.1) While the above informal definition of an undetectable subversion captures the intuitive idea behind subversion attacks nicely, it leaves open what kind of behaviour is detectable and what kind of behaviour is indistinguishable from a run of the non-subverted code.

In general, there are two easy and natural ways to leak sensitive information via subverted implementations. In the first kind of attack, the attacker restricts the attack to only replace the randomness used by the scheme.

One of the strongest attacks uses *rejection sampling*. In such an attack, the subverted implementation samples randomness $r_1, r_2, \dots$ until $c_i = \text{Com}(v, r_i)$ reveals some important information about sec , e.g., $\text{PRF}_k(c_i) = \text{sec}[0]$ for some PRF-key k known only to the attacker.

The advantage of this kind of attack is that the attacks often work in a somewhat black-box manner, as the subverted code will not care about the other inputs to the functionality. We note that nearly all known subversion attacks do fit into this category and only replace the randomness, e.g., [12, 28, 13, 8, 7, 43, 31, 5, 3, 53, 44, 54, 56, 29].

In the second kind of attack, attackers could also change the actual inputs of the functionality, e.g., by replacing one input by the secret.

To leak information about the secret sec , the attacker could simply commit to sec , i.e., produce a commitment $c = \text{Com}(\text{sec}, r)$ for a truly random r . Now, whenever the commitment c is opened, the receiver (as well as an eavesdropping attacker) will learn the secret sec .

Now, whether such an attack is undetectable highly depends on the situation. For some protocols, the secret might be a perfectly valid input or valid but uncommon enough to raise suspicion or it might even be invalid and thus not maintain the underlying functionality. For example, as discussed by Dodis et al. [33], such behaviour would usually be very obvious in message transmission protocols.

If the value v is usually a completely random value (such as those used in the GMW compiler [36]), the subverted code could produce a commitment $c = \text{Com}(\text{sec} \oplus k, r)$, where k is a key known only to the attacker. However, if v is a highly structured value (such as those in the MPC-in-the-Head zero knowledge proofs [40]), one would require a more advanced technique to embed the secret sec in such values.

Chakraborty et al. [26] decided to treat the second kind of attack behaviour as being undetectable. To counter these attacks, which they denote as *specious*, they require a way to also rerandomise the inputs (even in the ideal world). However, allowing such modifications, they focus on functionalities supporting random inputs. In contrast, we follow (and extend) the approach of Dodis et al. [33] and treat such a behaviour as detectable. We only allow the subverted code to modify the *randomness*, i.e., the attacker can influence the output distribution but the subverted code must at all times produce valid output. Note that this still gives a wide range of possibilities to the adversary, as they might, e.g., replace random nonces by pseudorandom ones, influence the output distribution of the protocols, or decrease the success probabilities of the protocol. Our attack scenario thus

does not capture the case where the replacement of an input would go unnoticed (which has not been used in any real-word subversion [33]). However, we also allow for attackers to, e.g., fix the randomness to a certain value, which might be very obvious and is thus not captured by the model of Chakraborty et al. [26]. Thus, our attacker models are incomparable. Furthermore, our design allows us to work with well-established classic ideal functionalities instead of providing an explicit *subversion interface*.

Protection Mechanisms (Section 4) In order to protect against subversion attacks, several different countermeasures have been proposed in the literature. The two most studied ones are *cryptographic reverse firewalls*, introduced by Mironov and Stephens-Davidowitz [46], and *watchdogs*, introduced by Bellare, Paterson, and Rogaway [8] (although the name was only coined later by Russell et al. [48]). Our second contribution is to model reverse firewalls in plain UC, which aim to actively remove the covert channel established by the subverted implementation leaking the secret. Intuitively, such a firewall is a device used by a cautious user who wants to protect their sensitive secret from exposure due to a subversion attack. The user thus installs the device close to their computer (e.g., by incorporating it into the router) such that the traffic observable by the attacker is available only *after* the data sent from the computer was passed to the firewall. The goal of the cryptographic reverse firewall is to remove the covert channel from the output of the user’s computer. Typically, this is done via the means of rerandomisation.

Suppose that the above commitment scheme Com is additively homomorphic, i.e., $\text{Com}(v, r) \oplus \text{Com}(v', r') = \text{Com}(v \oplus v', r \oplus r')$. If the user sends a commitment via its subverted implementation $c = \bar{\text{Com}}(v, r)$, this commitment is first given to the firewall before it becomes visible to the attacker. The firewall now samples a random string r' and computes $c' = c \oplus \text{Com}(0, r')$. As r' is sampled uniformly, c' cannot leak sensitive information via the used randomness.

In our model, such firewalls exist as separate parties in the real world. In UC-terminology, such firewalls are modelled as sub-parties of a main party, where the firewalls and main parties each have their own inputs and outputs from other protocol parties. Note that the adversary (the environment) cannot communicate directly with the firewall.

This is a contrast to the model of Chakraborty et al. [26], where the firewall is a direct part of the party to be protected. As we show that subversion attacks are preventable via firewalls in the real world, we do not need to model subversion in the ideal world, hence, these firewalls do not need to exist in the ideal world. This allows us to consider standard ideal functionalities in our setting, similar to the approach of Dodis et al. [33]. Again, this is contrary to the model of Chakraborty et al. [26], where the ideal functionalities need to have an explicit interface for the firewalls, called the *sanitisation interface*.

The firewalls have a number of important natural properties such as *functionality maintenance*—the firewall should not break the functionality of the protocol

run by unsubverted parties; *security preservation*—a subverted party should not be able to break the security guarantees of the protocol; and *exfiltration resistance*—a subverted party should not be able to leak any information related to the secret. A stronger version of the latter property is called *anti-signalling* which, intuitively, guarantees that even the attacker providing the subverted code cannot distinguish whether the party running behind the firewall uses the original code or the subverted code. Another very important property concerns the *transparency* of the firewall, i.e., whether the parties need to be aware of the existence of the firewall. Depending on the party, this transparency might be quite different. For a party $\mathcal{P}$ using a firewall RF , we want a form of *inner transparency* that guarantees that the party does not need to be aware of the existence of (possibly multiple) firewalls. A weaker form of this was introduced by Mironov and Stephens-Davidowitz as *stackability*, guaranteeing that a party could have arbitrarily many firewalls. Similar to the inner transparency, a party $\mathcal{P}'$ interacting with party $\mathcal{P}$ that has firewall RF installed should not need to be aware of the existence of RF . We call this *outer transparency*. Due to the existence of the sanitisation interface in the model of Chakraborty et al., $\mathcal{P}$ needs to be aware of RF and inner transparency is thus not achievable in their model.

Proving Security (Section 5) After we established our models for subversion and cryptographic firewalls, we also develop subversion-resilient protocols. To do so, we assume that we are given a classic protocol Π' , which does not use cryptographic firewalls and is thus (potentially) vulnerable to subversion attacks. This protocol securely UC-realises some ideal functionality $\mathcal{F}$. To protect Π' against subversion attacks, we now equip every party $\mathcal{P}_i$ with a firewall RF_i . This gives us a protocol Π . Now, to prove that Π UC-realises $\mathcal{F}$ under subversion-corruption via randomness replacement, we can show that it is sufficient to prove that all firewalls are transparent and anti-signalling and that they keep the correctness of Π' . This implies that Π securely UC-realises Π' and the transitivity of UC-emulation [17] thus implies that Π also UC-realises $\mathcal{F}$. Note that we ignore subversion corruptions in Π' and $\mathcal{F}$, but still provide subversion resilience. Hence, to prove security in our model, we can directly take and adapt existing secure protocols and only need to consider the properties of transparency, anti-signalling, and correctness. In a sense, we are able to transfer the proof technique of Mironov and Stephens-Davidowitz [46] to the UC setting that allows us to treat a transparent firewall as “external”. In contrast, Chakraborty et al. [26] needed to “reprove” the security guarantees of Π' when arguing about the security of Π as they bring subversion attacks to the ideal world.

Commitments and OT (Section 7) To show the flexibility of our approach, we instantiate two example functionalities, commitment schemes and oblivious transfer. We consider a commitment scheme due to Canetti, Sarkar, and Wang [21], which is also the basis of the subversion-resilient commitment scheme of Chakraborty et al. [26]. As explained above, we only need to show correctness, transparency, and anti-signalling and do not need to re-establish the UC-security of the protocol. Hence, our complete security proof for the commitment scheme is only a single page long. We also consider an oblivious transfer protocol due to [21] and, again,

only need to consider correctness, transparency, and anti-signalling. Our complete security proof is thus shorter than two pages. The question of the existence of such a concretely efficient subversion-resilient oblivious transfer protocol was raised by Chakraborty, Magri, Nielsen, and Venturi [26] and by Chakraborty, Ganesh, and Sarkar [24]. In the case of the non-plain UC model, this problem was independently solved by Chakraborty et al. [25]. We show that our model allows to present a much simpler and more efficient OT protocol in plain UC.

Malicious Firewalls (Section 8) At the first glance, transparency might look like a feature that is clearly nice to have, but not of particular importance for obtaining security guarantees. However, as we show in this paper, transparent firewalls are very useful to guarantee security in the case that party $\mathcal{P}$ is honest, but the firewall RF is (maliciously) corrupted. In the model of Chakraborty et al. [26], the firewall and the party are treated as a single party and hence, this single party needs to be treated as corrupted. Using a more fine-grained analysis, the authors note some security guarantees might be preserved in this setting and thus establish a new corruption model called *isolation*, but treat this as malicious corruption throughout their work. Hence, the installation of a firewall might actually *weaken* the security of the complete system.

In our model, the party and the firewall are two separate parties and we can thus analyse this important scenario of a corrupted firewall in more detail. Suppose that an honest party $\mathcal{P}$ using a corrupted firewall RF interacts with another party $\mathcal{P}'$ that can either be honest or corrupted. We first consider the security guarantees of $\mathcal{P}'$. Due to the outer transparency of the firewall, the party $\mathcal{P}'$ does not need to be aware of the existence of RF . From the perspective of $\mathcal{P}'$, the *combination* of $\mathcal{P}$ and RF can be seen as a single corrupted party and the active security of the underlying protocol thus gives the security guarantees of $\mathcal{P}'$. A similar argument is also used by Chakraborty et al.

Now, consider the security guarantees of $\mathcal{P}$. Due to the inner transparency of the firewall, the party $\mathcal{P}$ does not need to be aware of the existence of RF . Again, from the view of $\mathcal{P}$, we can thus treat the *combination* of RF and $\mathcal{P}'$ as a single corrupted party and the active security of the underlying protocol thus gives the security guarantees of $\mathcal{P}$. As described above, this argument does not work in the model of Chakraborty et al., as $\mathcal{P}$ needs to be aware of the existence of RF .

However, when taking a closer look at the definition of transparency, there is one problem left: As the firewall is allowed to send messages in the name of $\mathcal{P}$, a corrupted firewall can *impersonate* $\mathcal{P}$ due to the outer transparency. Note that this is a separate issue to attacks that embed information and, as long as the party is honest, no sensitive information will be leaked. This impersonation problem is somewhat unavoidable due to the requirement of outer transparency and, thus, already applies to the original model of cryptographic firewalls due to Mironov and Stephens-Davidowitz [46] as well as Chakraborty et al. [26]. For protocols not relying on authentication, this is not a problem (and only such protocols were studied previously such as commitments or key exchange). However, when considering more general protocols, such behaviour might actually weaken the security guarantees. To handle this issue explicitly, we introduce a

new ideal functionality, called $\mathcal{F}_{\text{chAUTH}}$ which explicitly covers the possibilities of impersonation attacks by a corrupted firewall. We also show two realisations of this functionality: A very efficient one for our commitment protocol based on signatures and a more general version using zero-knowledge proofs for our oblivious transfer protocol.

1.3 Related Work

The concept of reverse firewalls was first introduced by Mironov and Stephens-Davidowitz [46]. They applied it to different primitives in the semi-honest setting against subversions that could also alter the functionality of the underlying primitive. Building upon this, Dodis et al. [33] considered protocols also in the malicious setting, but only against subversions that were functionality-maintaining (due to some strong impossibility results). The primitives secured by firewalls in these works and follow-up works range from oblivious transfer [46], garbled circuits [46], generic constructions against passive attackers [46], CPA-secure encryption [33], key agreement [33], CCA-secure encryption [33], interactive proof systems [35], secure channels [14], and actively secure MPC [22, 23] to oblivious transfer extensions [24]. Li et al. [42] were the first to consider reverse firewalls in a UC-context, but deviate by allowing for an “alarm” signal. The model of Chakraborty et al. [26] does not allow for such an “alarm” and is thus somewhat closer to the original definition of Mironov and Stephens-Davidowitz. A comparison of our model with the model of Chakraborty et al. [26] can be found in Appendix A. Concurrently and independently from this work, Chakraborty et al. [25] presented a subversion-resilient oblivious transfer and password-authenticated key exchange in the model of Chakraborty et al. [26].

While cryptographic reverse firewalls have many advantages, other approaches to prevent subversion attacks exist.

Mediators: A similar approach to reverse firewalls was presented by Alwen, shelat, and Visconti [2]. They make use of a so-called *mediator*, who is also allowed to rerandomise messages, to obtain so-called *collusion-free* protocols, as introduced by Lepinski, Micali, and shelat [41]. However, in contrast to the reverse firewalls, such a mediator is able to intercept the messages of *all* parties (not only those that deployed the firewall) and, furthermore, it can actively exchange messages with all parties in an arbitrary order. Hence, such a mediator is a much stronger concept than reverse firewalls. We refer the reader to Mironov and Stephens-Davidowitz [46] for a more detailed comparison.

Other UC firewalls: Li et al. [42] were the first to consider reverse firewalls in a UC-context. However, their definition strongly differs from the original definition due to Mironov and Stephens-Davidowitz, as their firewalls can detect a subversion. In this case, the firewall “alarms”, directly halting the complete protocol. In some sense, their model thus uses a mix of reverse firewalls and watchdogs (discussed below). Furthermore, they consider their firewalls to be uncorruptable and make use of random oracles to make the firewalls somewhat deterministic.

Watchdogs: The notion of a *watchdog* was introduced by Bellare, Paterson, and Rogaway [8]. A watchdog can either be *offline* or *online*. An offline watchdog is given the (possibly subverted) implementation along with an honest reference implementation and before the user uses the (possibly subverted) implementation, the watchdog can perform certain black-box tests to find differences between the two implementations. If it can't find any difference, the user is free to use the implementation. An online watchdog can additionally also observe the complete communication while the implementation is in use in order to detect deviant behaviour. Watchdogs have also been used to secure different primitives such as one-way-permutations [48], pseudorandom generators [48], randomness generators [11, 48], public-key encryption schemes [11, 50], authenticated encryption [10], random oracles [49], and signature schemes [30, 48, 9].

Alternative Countermeasures: Both Fischlin and Mazaheri [34] and Abdolmaleki et al. [1] assume the existence of a non-compromised first phase where the parties can freely exchange information. Fischlin and Mazaheri use this phase to exchange an honestly signed message which then is used to create subversion-resilient signatures. Abdolmaleki et al. use this first secure phase to exchange initial randomness from which they can derive non-subverted randomness for the remaining zero-knowledge protocol. Finally, Badertscher et al. [6] also consider a kind of subversion attacks in a UC-setting, but, in contrast to nearly all of the aforementioned papers, their implementation does not try to be undetectable. They show in such a scenario, how updates can be performed that return the implementation to a non-subverted state and consider digital signatures and zero-knowledge proofs.

2 Preliminaries

In this section, we describe the needed preliminaries and notations used throughout this work.

Notation The term PPT will be used for Probabilistic Polynomial Time, i.e., efficient randomised algorithms. Computational indistinguishability is denoted by $\approx$.

Glossary Throughout this paper, we use the following notations unless stated otherwise:

$\mathcal{P}$	a party
$\mathcal{P}'$	a second party
RF	a firewall which is itself also a party
$RF \circ \mathcal{P}$	a composed party consisting of a party $\mathcal{P}$ and a firewall RF
$\hat{\mathcal{P}}$	an incorruptible party $\mathcal{P}$
$\bar{\mathcal{P}}$	a subvertable party $\mathcal{P}$
π	a code

Π a protocol

2.1 Cryptographic Primitives

In this work, we will consider multiple cryptographic primitives. Here, we only give an informal overview and will present the needed technical definitions throughout the paper.

Commitment: A *commitment scheme* consists of a single PPT algorithm Com that is given some value $v \in \{0, 1\}^*$ and produces a commitment c . The commitment is called *hiding* if no attacker can distinguish between commitments of v and commitments of another value v' , even if v and v' are provided by the attacker. The commitment is called *binding* if it is infeasible to find two random strings r and r' and two different values v and v' such that $\text{Com}(v, r) = \text{Com}(v', r')$.

Oblivious Transfer: An *oblivious transfer* (OT) consists of two PPT algorithms $(S, \mathcal{R})$. The *sender* S knows two messages m_0 and m_1 and the *receiver* $\mathcal{R}$ knows a bit b . After interaction between S and $\mathcal{R}$, the receiver should obtain the message m_b without learning anything about the other messages m_{1-b} . Furthermore, S should not learn anything about the bit b .

Signature Schemes: A *signature scheme* consists of three PPT algorithms Kgen , Sign , and Vfy . The *key generation algorithm* Kgen is given a security parameter 1^κ and produces both the verification key vk and the signing key sk . The *signing algorithm* Sign is given the signing key sk and a message $msg \in \{0, 1\}^*$ and produces a signature σ . Finally, the *verification algorithm* Vfy is given the verification key vk , a message msg , and a signature σ and outputs a bit $b \in \{0, 1\}$. For all outputs σ of $\text{Sign}(sk, msg)$, we need that $\text{Vfy}(vk, msg, \sigma) = 1$. A signature scheme is called *universal unforgeable* if all attackers with oracle access to $\text{Sign}(sk, \cdot)$ are not able to produce a valid pair (m, σ) (except for those provided by the oracle).

Zero-Knowledge proofs: A *zero-knowledge proof* for a language $\mathcal{L} \subseteq \{0, 1\}^*$ consists of a pair of PPT algorithms $(\mathcal{P}, \mathcal{V})$. Both parties know some *instance* $x \in \{0, 1\}^*$ and the *prover* $\mathcal{P}$ knows a witness proving that $x \in \mathcal{L}$ and wants to convince the *verifier* $\mathcal{V}$ that $x \in \mathcal{L}$. A pair $(\mathcal{P}, \mathcal{V})$ is called *complete*, if an honest prover can (nearly) always convince a verifier and *sound* if a dishonest prover can (nearly) never convince a verifier. Finally, the system is said to have the *ZK property* if even a dishonest verifier does not learn any information about the witness held by the honest prover.

2.2 Universal Composability

The traditionally employed game-based security notions have the disadvantage that they usually cannot capture *compositions*. To still guarantee security in such quite involved situation, Canetti [18, 16] introduced the *Universal Composability* (UC) model. To formalise security in the UC model, one defines an *ideal functionality* $\mathcal{F}$ that captures the ideal behaviour of the primitive. To *implement* such

an ideal functionality, one formalises a protocol Π and then shows through a simulation that a real world execution of this protocol is indistinguishable from a run of the ideal functionality. More explicitly, the distribution of all outputs of both protocol runs by any input from the PPT $\mathcal{Z}$ is indistinguishable for the environment $\mathcal{Z}$ and we will say the protocol securely UC-realises the ideal functionality.

Every party in a protocol consists of a *body* and a *shell*. The body runs the part of the protocol which is dedicated to that party, i.e., all computations. It receives all inputs via the shell and gives all output to the shell. One task of the shell is to transfer the in- and outputs to the respective parties. The main task of the shell is to control the model-related instructions. It handles the communication and party-corruption mechanisms. Therefore, the shell does not contain the actual program code but modelling code. For more details we refer to Section §5.1 in [18, 16].

Regarding the corruption behaviour of a party, it is worth mentioning that when we talk about an *incorruptible* party $\hat{\mathcal{P}}$, it means that the shell of this party ignores all instructions to corrupt the body. This results in an honest behaviour of the party throughout the whole run.

More detailed, we are given n parties $\mathcal{P}_1, \dots, \mathcal{P}_n$ which are interactive Turing machines (ITMs) with codes $\pi_1, \dots, \pi_n$. These parties can be *corrupted* by the attacker $\mathcal{A}$ that sends certain corruption messages to the parties to corrupt them, e.g., maliciously. The corrupted parties are now somehow controlled by $\mathcal{A}$, although their behaviour might be restricted by their corruption type.

To achieve security under general composition, the framework introduces a distinguisher $\mathcal{Z}$, called the *environment*, which can interact with the executions by, e.g., providing inputs and interacting with the attacker $\mathcal{A}$. We denote a real-world execution of the protocol Π with the adversary $\mathcal{A}$ and the environment $\mathcal{Z}$ using the security parameter κ and the randomness r by $\text{EXEC}_{\Pi, \mathcal{A}, \mathcal{Z}}(\kappa, r)$. This execution is compared with a run of the ideal functionality $\mathcal{F}$ where all parties are dummy parties controlled by the environment $\mathcal{Z}$. The goal is now to construct a *simulator* Sim that can interact with the ideal functionality such that the real execution is indistinguishable from the ideal execution $\text{EXEC}_{\mathcal{F}, \text{Sim}, \mathcal{Z}}(\kappa, r)$.

Formally, this is defined as:

Definition 1 (UC-security). *A protocol Π UC-realises an ideal functionality $\mathcal{F}$ (in symbols $\Pi \geq_{\text{UC}} \mathcal{F}$) if for all PPT attackers $\mathcal{A}$, there exists a PPT simulator Sim such that for all PPT environments $\mathcal{Z}$, we have that $\text{EXEC}_{\Pi, \mathcal{A}, \mathcal{Z}}(\kappa, r)$ is computationally indistinguishable from $\text{EXEC}_{\mathcal{F}, \text{Sim}, \mathcal{Z}}(\kappa, r)$ (with respect to the security parameter κ). Here, r is drawn randomly.*

Note that if both executions are real world protocols, a simulator showing the indistinguishability lets one protocol *UC-emulate* the other one.

All parties can communicate with other ideal functionalities $\mathcal{F}'$. A protocol designed to having access to $\mathcal{F}'$ is said to be in the $\mathcal{F}'$ -hybrid model. The *composition theorem* now allows to replace this ideal functionality $\mathcal{F}'$ by any protocol Π' UC-realising $\mathcal{F}'$ while keeping all of the security guarantees preserved.

In the following, we will explain some ideal functionalities which will be used throughout this paper.

Definition 2 (The Ideal Functionality $\mathcal{F}_{CRS}$, adapted from [19]).

$\mathcal{F}_{CRS}$

The ideal functionality $\mathcal{F}_{CRS}$ models a common reference string that is accessible by all parties. $\mathcal{F}_{CRS}$ proceeds as follows, when parametrised with a distribution D .

- *When activated for the first time on input $(\text{value}, \text{sid})$ by a party $\mathcal{P}$, choose a value $d \leftarrow D$, and generate a public delayed output $(\text{value}, \text{sid}, d)$ for $\mathcal{P}$.*
- *Answer subsequent value inputs from parties $\mathcal{P}'$ by generating a public delayed output $(\text{value}, \text{sid}, d)$ for $\mathcal{P}'$.*

Definition 3 (The Ideal Functionality $\mathcal{F}_{AUTH}$, adapted from [17]).

$\mathcal{F}_{AUTH}$

The ideal functionality $\mathcal{F}_{AUTH}$ models an authenticated channel between a sender S and a receiver $\mathcal{R}$.

- *Upon invocation, with input $(\text{send}, \text{sid}, S, \mathcal{R}, m)$ from S , send backdoor message $(\text{send}, \text{sid}, S, \mathcal{R}, m)$ to the adversary.*
- *Upon receiving backdoor message (OK, sid) : If not yet generated output, then output $(\text{sent}, \text{sid}, S, \mathcal{R}, m)$ to $\mathcal{R}$.*
- *Upon receiving backdoor message $(\text{corrupt}, \text{sid}, m', \mathcal{R}')$, record being corrupted. Next, if not yet generated output then output $(\text{sent}, \text{sid}, S, \mathcal{R}', m')$ to $\mathcal{R}'$.*
- *On input $(\text{reportcorrupted}, \text{sid})$ from S : If corrupted, output **yes** to S , else output **no**.*
- *Ignore all other inputs and backdoor messages.*

3 Subversion Attacks and Countermeasures in UC

The main objective of our paper is to study the subversion of a party. A *subversion* corruption allows an adversary—which is hence called a *subverter*—to change the code π to a *subverted* one $\bar{\pi}$ of a then *subverted*, i.e., subversion corrupted, party. Unlike other corruption behaviours like malicious or semi-honest corruptions, the adversary has no further access to the party apart from this code replacement. Other information of the party such as a specific secret sec is, hence, not inferable from the subverter directly, but might be extractable through the subverted code. Again, this secret may or may not contain information which, when learned by the adversary, could break the functionality of the protocols. An example secret could be a seed allowing to derive some used key.

3.1 Modelling Subversion Corruptions in UC

In the framework of Universal Composability [17], we model a protocol run via the interaction of several parties $\mathcal{P}_1, \mathcal{P}_2, \dots, \mathcal{P}_n$. We assume that each party $\mathcal{P}_i$ has some additional sensitive information called sec_i . Each $\mathcal{P}_i$ runs part of the implemented protocol Π in its body, namely π_i . An adversary $\mathcal{A}$ has the goal of gaining information about the secrets of the parties. Therefore, it chooses a set of parties $C \subseteq \{1, \dots, n\}$ that it wants to *subvert*. To do so, it replaces the code π_i of each party $\mathcal{P}_i$ with $i \in C$ by a subverted version $\bar{\pi}_i$. Remark that the code π_i , or $\bar{\pi}_i$ respectively, has access to sec_i (which might be the empty string if no secret is used). Then, $\mathcal{A}$ chooses another set of parties C' with $C \cap C' = \emptyset$ to corrupt maliciously or semi-honestly. Now, the parties interact following protocol Π and the adversary tries to gain information about all secrets sec_i with $i \in C$. Informally, the subversion of a single party can be modelled as in Fig. 1.

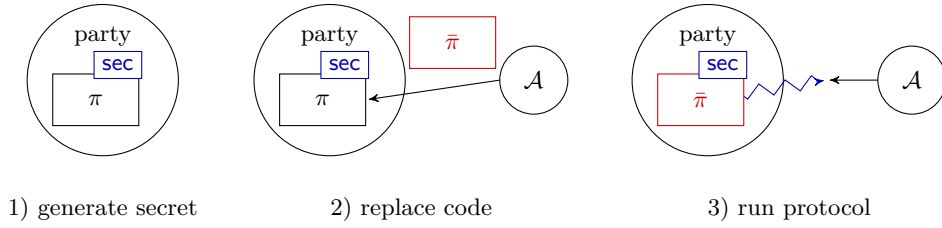


Fig. 1: The adversary can replace the code π of a party with a subverted one $\bar{\pi}$ which can then leak information about the secret to the adversary $\mathcal{A}$.

In the modelling of UC, the environment $\mathcal{Z}$ plays the role of all probabilistic polynomial-time (PPT) adversaries. It can leverage different corruption parameters discussed below which allow to insert further messages, receive internal state messages, and listen to the communication channel depending on the given communication setting. Despite all that, the environment should not be able to distinguish two protocols where one is the idealised goal and the other describes reality as closely as possible.

To model subversions in the framework of UC, the corruption interfaces have to be expanded to handle this new type of corruption. Chakraborty et al. [26] have already presented an adaptation of the UC model that allows modelling subversion attacks and their protections. The authors resorted to only considering undetectable, so-called *specious* subversions and only quantify over specious environments and adversaries. In contrast, we quantify over all PPT environments and introduce a corruption model extending the existing corruption models in the *plain* UC framework by Canetti [17]. This allows reusing well-known important results such as the composition theorem [18] without the need to prove them anew as was necessary for [26]. An additional advantage of this model is that because the model is only extended but not restricted in any way, one can analyse subversion corruption conjointly with other corruption settings.

To analyse a protocol, we give a formal model of the real world protocol which includes both the possible attacks in the form of subversion corruptions by the environment and protective reverse firewalls for each main party. How the firewalls are used as a countermeasure against subversion attacks is discussed in Section 4. However, this newly inserted sub-party will also open up a new attack vector during the protocol run. As, in reality, a reverse firewall is an additional hardware module, it itself can be manipulated by design or contain an implementation error. To formalise some kind of malicious behaviour, we, hence, give the environment the option to corrupt the firewall. Informally, the subversion corruption of a party Alice with its firewall RF_A is modelled as depicted in Fig. 2, which also shows the insertion of a secret sec by the environment $\mathcal{Z}$.

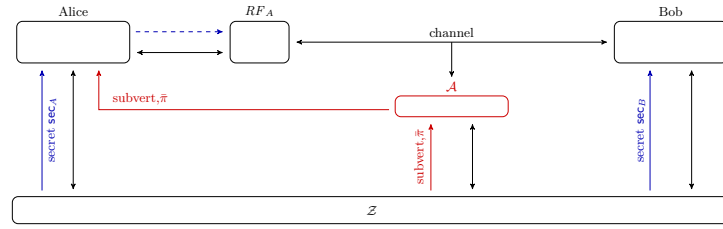


Fig. 2: Informal visualisation of a subversion corruption in the real world in the UC model. Information about the secret might be leaked to the firewall (represented by $-->$). The goal of the firewall is to output messages to the channel that no longer contain this leakage. The leakage of Bob is omitted.

Secret To provide an easier understanding of subversion as a general attack, we claim that every party has a secret sec which we model explicitly. A secret could be some key material from a higher-level protocol, a public-key infrastructure certificate, or the long-term secret family recipe of grandmother’s famous apple pie. To model the subversion attack goal, every main party $\mathcal{P}$ is initialised with an empty secret tape, part of its internal state, similar to the randomness tape. This tape is used to store a secret sec_P that is inserted by the environment via the **secret**-interface at any time during the protocol run. The interface mechanism of the shell of $\mathcal{P}$ works such that on input $(\text{secret}, \text{sid}, \text{sec}_P)$ from $\mathcal{A}$, sec_P is written on the secret tape (see below for more information). We will omit the index and simply write sec if the main party is clear from context. As the adversary has no access to the subverted party during the protocol run, it needs access to the communication generated by the party to benefit from the subversion corruption. This can be achieved either by eavesdropping on the communication channel, which can then be *authenticated* at best, or by corrupting the communication partner of the subverted party.

Remark 1. The explicit secret tape does not change the machinery system of the UC model as it is seen as one part of the internal state tape. Afterwards, the

body only has read-only access to this tape part. This modelling is similar to the one of the randomness tape.

Undetectable Subversion While, in theory, an adversary could arbitrarily subvert a code, some of these changes are very obvious and would raise suspicion. An adversary is thus naturally inclined to only subvert in a way that most communication looks inconspicuous, but capturing this type of subversion is not that simple. Chakraborty et al. [26] tackle this by introducing the definition of *specious* subversions, meaning that no PPT test can distinguish between black-box access to a party running the honest protocol and a party running the subverted protocol except with negligible probability. Their test is not given in a UC definition and, while it helps in understanding this kind of subversion attacks, it is not directly transferable to a corruption behaviour in UC. This essentially results in them only allowing “correct” subversions as well as a sparse amount of trigger attacks. Note that this notion of correctness also encompasses the adversary changing the inputs of the functionality, e.g., by replacing one input by the secret `sec`. Depending on the protocol, using the secret `sec` as an input might be perfectly valid. It could, however, also raise suspicion in case it is a valid but uncommon or even an invalid input, the latter of which would not even maintain the underlying functionality. By allowing to change the inputs and declaring such an attack as being specious, Chakraborty et al. [26] need a way to also rerandomise the inputs (even in the ideal world) and either restrict to functionalities that use random inputs or need to trust the firewall to not change the input (a combination of this can be seen in their GMW compiler).

In contrast, we approach the problem of describing undetectable subversions from a different point of view. Instead of allowing subversions based on how the output distribution of the subverted code looks like, one can restrict subversions based on the changes that are done by the adversary. We choose to focus on a natural way of leaking sensitive information via subverted implementations where the randomness used by the scheme can be changed. By limiting the attack to only this, the output of the subverted implementation is guaranteed to always be correct and, hence, a mere check for correctness of the implementation is not sufficient to detect such a subversion. To the best of our knowledge, there is no paper not satisfying this assumption and there is a variety of subversion attacks that do, e.g., [12, 28, 13, 8, 7, 43, 31, 5, 3, 53, 44, 54, 56, 51]. This also includes all known real-world compromises that were found (see, e.g., Dodis et al. [33, Section 1.1] for a detailed list of such compromises).

Note that only allowing the adversary to change the randomness means that we do not allow changing the inputs of the functionality and such a behaviour is treated as *detectable*. Hence, we do not need to bring the subversion attack vector to the ideal world. We will show that with the help of a firewall, one can already prevent subversion corruptions in the real world by showing indistinguishability with respect to classic real-world protocols and, hence, to well-known ideal UC functionalities. Unlike the approach of Chakraborty et al. [26], we can, therefore, continue to work with the well-established classic ideal functionalities instead of

providing an explicit *subversion interface*. However, there are also subversions that we treat as *undetectable* that [26] do not allow. For example, the adversary could change the randomness such that the same constant is used throughout multiple protocol runs. This is clearly not specious, as a test could distinguish such a code from an honest one. Therefore, while the two approaches fall in the category of correct subversions, they are not directly comparable.

As we only consider attacks that change the randomness, we can model them in the plain UC framework as follows: The shell of a party only accepts subversion corruption messages with codes $\bar{\pi}$ which replace at most the randomness function with an adversarially-chosen function. This function has read-only access to the randomness tape and the whole state of the party including its secret sec , meaning that it can even take the sec as in- and output. The randomness function will then be inserted by the shell into the body such that each call to this so-called `getRandomness`-function is replaced by a call to this `getRandomness`-function of $\bar{\pi}$. However, no other change of the code run by the body is allowed. The function is inserted in the body such that every read access to the randomness tape is replaced by a read access to this adversarially-chosen randomness function. Note that the shell can easily verify that indeed only the randomness function was replaced. Therefore, we say that $\bar{\pi}$ is only valid if the shell of a subverted party can perform such a check of the validity.

Definition 4 (Subversion via randomness replacement). *Let π be the original code and $\bar{\pi}$ the subverted code, where the `getRandomness`-function is replaced. Let Π (Π' , resp.) be the protocol which runs with the code π ($\bar{\pi}$, resp.). A subversion is detectable if there exists a PPT environment $\mathcal{Z}$ for which $\text{EXEC}_{\Pi, \mathcal{A}, \mathcal{Z}}(\kappa, r)$ and $\text{EXEC}_{\Pi', \mathcal{A}, \mathcal{Z}}(\kappa, r)$ are distinguishable, where $\mathcal{A}$ is the dummy adversary, κ the security parameter, and r the uniformly random chosen randomness.*

From now on, we will refer to the subversion via randomness replacement simply by *subversion*. Whenever we will talk about other types of subversions, we will explicitly say so.

3.2 Preventing Subversion Attacks in UC Using Reverse Firewalls

After formalising the subversion corruption model and the undetectable subversions considered in this work, we now focus on how to protect protocols against such attacks. While different techniques exist in the literature, we focus on the seminal standard of *reverse firewalls* introduced by Mironov and Stephens-Davidowitz [46].

Introduction to Reverse Firewalls Informally, a reverse firewall $\text{RF}_{\mathcal{P}}$ (in short only called firewall or RF) is a device additionally installed by a party $\mathcal{P}$ to *protect* the output of their (possibly subverted) implementation from leaking information about the secret, i.e., *signalling*. The firewall obtains all outgoing communication from $\mathcal{P}$ and all ingoing communication to it. Note that while they are two separate identities, we will call $\mathcal{P}$ a (main) party of a protocol Π

and $RF_{\mathcal{P}}$ the sub-party of $\mathcal{P}$. If the main party is clear from context, we will simply write RF . A firewall RF is modelled as a sub-party of main party $\mathcal{P}$ in the real world as visualised in Fig. 2. Every main party might have one (or more) firewalls but is not required to have one. In this work, we will analyse the security of party $\mathcal{P}$ gained by its RF in the case of subversion corruption. Therefore, we always have the composed party of $\mathcal{P}$ and RF in mind which we denote by $RF \circ \mathcal{P}$ where all communication is routed via RF . Hence, firewalls communicate via an immediate channel with the respective main parties and, for the communication onward to the network channel, pass on the message after modifying part of the content of the message without modifying the header. The usual setting, which we also use, is the idea that the firewall is somewhat *close* to the party deploying it in such a way that the communication between the party and their firewall can not be eavesdropped.

Remark 2. An *immediate* channel is a direct communication between a party and its firewall without any notification about the transmission passing via an adversary of the protocol. In a modelling sense, it corresponds to a direct transmission of any message from the shell of the party to the shell of the recipient party, i.e. its firewall. The shell of a main party needs to know if there is a firewall, and, continually, the shell of a firewall should be aware of the next firewall. Thus, each shell needs to know the start- and endpoint of the next communication partner, though this is always the case within the UC framework. Keeping in mind that the shell is responsible for the modelling code of a protocol, one can easily see that the body of a party and/or a firewall does not need to be aware of the existence of a firewall. This gives a good model of how messages are routed in a real-life network that passes messages/packets along a sequence of routers. Here, each device in a network only needs to know the address of its own router. Note that RF sends messages in the name of $\mathcal{P}$ which could be used as an impersonation attack by a malicious RF . See Section 8 for a detailed discussion on this. Apart from the interfaces for the input from its main party and the network channel, no further inputs from the outside are modelled/accepted, i.e., the environment $\mathcal{Z}$ cannot give further input to the firewall.

Subversion Corruption Handling in the Real World As mentioned before, we handle subversion corruptions in such a way that they are already addressed in the real world and not in the ideal world. However, while firewalls allow us to protect parties against subversion attacks, they also bring along possible vulnerabilities, as they can be corrupted by an adversary as well.

Our goal consists in precisely analysing what kind of security guarantee is preserved if even the firewall of a party is corrupted in a certain way. Therefore, we give all environments the additional power of specifying in the corruption parameter not only how the main party is corrupted but also how a potential sub-party should be corrupted. That means that the environment/adversary sends a corruption message with $(\text{corrupt}, \text{sid}, cp)$ where the corruption parameter cp includes information of the main party and its potentially existing sub-parties, e.g., $cp = (\text{subversion} : \bar{\pi}; RF \text{ semi-honest})$. We assume that the shell of a

party that does not have a firewall simply ignores the part of cp that does not concern its party. This corruption separation allows us to analyse which kind of security is achievable when, e.g., the firewalls are corrupted. The partial corruption, i.e., corrupting sub-parties of a differently corrupted main party, in the classic UC framework can lead to difficulties as was shown in [39]. However, we want to especially analyse remaining security guarantees while corrupting a main and a sub-party differently. Analysing realistic partial attacks in UC is, e.g., also researched by [15]. For a general understanding of corruption modelling in the UC-setting including the handling via the Corruption Aggregation ITI, we adapt [17, Sec 7.1].

In this work, we consider corruption parameters $cp = (cp(\mathcal{P}); RF\ cp(RF))$ with all kinds of combinations of $cp(\mathcal{P}) \in \{\text{honest}, \text{malicious}, \text{subversion} : \pi\}$ and $cp(RF) \in \{\text{honest}, \text{semi-honest}, \text{malicious}\}$. Note that $cp = \text{honest}$ means that this party is not corrupted. The different possible corruption parameter combinations of main protocol parties and their firewalls will be translated into a composed classic corruption behaviour. This will then be the behaviour of the corruption parameter for the corresponding main party in a known base protocol which does not have a firewall as sub-party. An overview of the corruption behaviour is given in Table 2 for the corruption transition of a main party $\mathcal{P}$ and its sub-party RF in a protocol Π to its composed classic corruption behaviour. We achieve the classic corruptions of honest, semi-honest and malicious corruptions.

Table 2: Different corruption combinations and their composed classic corruption behaviour. The lines in grey differ from the table given in [26].

Party	Firewall	Composed Classic Behaviour
honest	honest / semi-honest	honest
honest	malicious	malicious/honest, see Section 8
malicious	honest/semi-honest/malicious	malicious
subversion	honest	honest
subversion	semi-honest	semi-honest
subversion	malicious	malicious

The validity of the results of this table is analysed in Section 5. For this work, the most relevant cases are the ones where the main party $\mathcal{P}$ is subversion corrupted. Interestingly, in the case of a subversion corrupted party with its firewall being only semi-honest, we can show that we achieve a semi-honest composed classic corruption behaviour whereas [26] consider this to be malicious behaviour. An additional interesting result of this work shows that when adapting the protocol, one can hope to achieve an honest composed classic corruption behaviour even if the main party is honest but the firewall behaves maliciously.

A subversion corruption allows an adversary to replace the code run by the body of a party. The environment $\mathcal{Z}$ sends a subversion corruption mes-

sage **subvert** including a code $\bar{\pi}$ to a party $\mathcal{P}$. Upon receiving **subvert** = (**corrupt**, sid , (**subversion** : $\bar{\pi}$; RF $cp(RF)$), the shell of party $\mathcal{P}$ checks if $\bar{\pi}$ is a valid subversion code, i.e., only the **getRandomness**-function is replaced, else the message is ignored. The shell then replaces the randomness function in the body by the adversarially-chosen **getRandomness**-function. Further corruption messages are ignored by the shell of the party. Apart from replacing the code in the body, a subversion corrupted party behaves as an honest party. Note that the provided code might be subverted but does not have to be. Hence, even if $\bar{\pi} = \pi$, we will talk about a subversion taking place. The detailed description of the party's shell code is visualised below. Prior to the randomness function replacement, the shell has to inform the Corruption Aggregation ITI about the corresponding corruption type. To determine the appropriate corruption parameter, the shell then parses the corruption parameter string cp to get the corruption parameter for the main party $\mathcal{P}$ and its firewall RF and follows the transition table shown in Table 2 to get the composed corruption parameter $cp(RF \circ \mathcal{P})$ which it sends to the Corruption Aggregation ITI. By this, the Corruption Aggregation ITI (and therefore the environment $\mathcal{Z}$) will be informed about the composed corruption behaviour in all cases. As one can see, the firewall is a self-contained identity which can be corrupted in a certain manner but as it is a sub-party, it will always behave in a composed setting with the main party. After the activation returns from the Corruption Aggregation ITI, the shell continues according to the corruption parameter $cp(\mathcal{P})$. When the body sends a message to the communication channel, i.e., $\mathcal{F}_{AUTH}$, the shell directs the message to its RF with the respective corruption parameter $cp(RF)$. The shell of the firewall behaves according to the corruption parameter without informing the Corruption Aggregation ITI. The detailed description of firewalls shell code is also visualised below.

Code of shell of party $\mathcal{P}$ regarding sec insertion

- On (**secret**, sid , **sec**) from $\mathcal{A}$, write **sec** on *secret tape*.

Code of shell of party $\mathcal{P}$ regarding corruption handling

- Upon invocation, send (**request-corruption**, sid) to $\mathcal{A}$.
- On next activation; on (**corrupt**, sid , cp) from $\mathcal{A}$:
 - If $cp(\mathcal{P}) = (\text{subversion} : \bar{\pi})$ and if $\bar{\pi}$ is no valid subversion code:
 - set $cp(\mathcal{P}) := \text{honest}$.
 - If $cp(RF) \notin \{\text{honest}, \text{semi-honest}, \text{malicious}\}$:
 - set $cp(RF) := \text{honest}$.
- Determine $cp(RF \circ \mathcal{P})$ following transition table Table 2 and send (**corrupt**, sid , $cp(RF \circ \mathcal{P})$, $\mathcal{P}$) to Agg-ITI.

- On activation return by Agg-ITI, proceed according to the respective specific corruption type, e.g., described in Section 7.1.1 in [17].
 If $cp(\mathcal{P}) = (\text{subversion} : \bar{\pi})$:
 replace $\bar{\pi}$ in the respective place in code of body and continue.
- $\vdots$
- Ignore further corruption messages.
- On instruction by body to send input to $\mathcal{F}_{\text{AUTH}}$, send input to RF with the respective corruption message $(\text{corrupt}, sid, cp(RF))$.

Code of shell of sub-party RF of $\mathcal{P}$ regarding corruption handling

- On invocation by $\mathcal{P}$ with input msg and corruption message $(\text{corrupt}, sid, cp(RF))$:
 $\backslash\backslash$ if RF is invoked by another party, then it first invokes its main party $\mathcal{P}$.
 continue according to the respective specific corruption type, e.g., described in Section 7.1.1 in [17].
 $\backslash\backslash$ do not inform the Agg-ITI.
- On instruction by body to send input to $\mathcal{F}_{\text{AUTH}}$, send input to $\mathcal{F}_{\text{AUTH}}$ in the name of $\mathcal{P}$.

Subversion Corruption Handling in the Ideal World To emphasise that subversion attacks are attacks against implementations, our goal is to only consider them in the real world. Therefore, given a standard protocol, we want to prove that its subversion resistance can already be analysed in the real world. We will show in Section 5 that a real world protocol with subversion corruption and a protection layer consisting of reverse firewalls is indistinguishable from a base protocol that is already proven to be secure in the non-subversion setting. A visualisation of the different protocols and the steps to show the indistinguishability is visible in Fig. 4. By already getting rid of the subversion attack in the real world, we enforce that subversion corruptions are ignored in the ideal world as they are not necessary there. As the adapted real world protocol (a) in Fig. 4 should be indistinguishable from the base protocol (b) in Fig. 4, the environment $\mathcal{Z}$ must not know if a RF exists. Therefore, $\mathcal{Z}$ does not corrupt RF directly and the corruption of RF is not reported to the Corruption Aggregation ITI.

As mentioned in Section 3.1, we assume that every party has a secret, which is modelled by giving all main parties an empty initialised tape where the environment can insert a secret $\text{sec}_{\mathcal{P}}$ via a specific interface. Such an adaptation of the model is not only necessary in the adapted real world protocol but also in every standard real world protocol as well as every ideal functionality. However, note that this is only a modelling artefact due to the corruption handling: The

secret must be inserted such that we can UC-simulate the case of a subverted party and a firewall that is semi-honestly or maliciously corrupted. In this case, the firewall would simply leak the secret to the environment and, hence, we model the secret explicitly throughout all protocols.

One could ask how one can see that the ideal functionality still preserves subversion resilience. Subversion corruptions are simply ignored by the ideal functionality such that we can still make use of well-established ideal functionalities. These are then indistinguishable from the standard real world protocol and in turn from the adapted real world protocol which allows subversion corruptions by the environment and protects against them via a reverse firewall.

Remark 3. In this paper, we focus on two-party computation. This allows an easier explanation of our model as, here, the interaction between one party using a firewall and “the outside” can be represented by only communicating with one partner. However, the model is also extendable to multi-party computation as we only look at the plain UC model here. See Section 3.3 for more details about the considered corruption.

3.3 More Details on Corruptions Considered in This Paper

Throughout this work, we consider the setting of static corruptions. Whether a party is corrupted or not is controlled via a corruption flag that is set if a corruption is valid; subsequent corruption commands are then ignored. We omit writing the flags explicitly for better readability.

Finally, we only consider attacks captured by the classic UC model. Different attacks using, e.g., the timing behaviour or other physical properties of an implementation are thus out of scope. However, we expect that our model could easily be integrated into different UC extensions capturing such properties such as the model of Canetti et al. [20].

Remark 4 (Comparison of Subversion and Semi-Honest Corruptions). In a *subversion* corruption, the adversary can choose the behaviour of the party in the beginning with knowledge about neither the input values nor the future states of the computation. This includes the activation status implying that the subverter cannot halt the party from the outside. In contrast, a semi-honest corruption allows learning all these information but gives no power to change the behaviour. In theory, it might thus be possible to combine the two corruption models to achieve semi-honest subversion corruptions. However, this would give an adversary direct access to the secret, making the whole goal of leaking the secret obsolete.

4 Properties of Reverse Firewalls

In the previous section, we introduced the corruption model and the notion of reverse firewalls, but have not yet described how to actually use such firewalls to protect implementations. In this section, we will formalise the properties of reverse

firewalls that are needed to guarantee security. Due to the immense strength of general subversion attacks, it is fairly easy to see that certain subversion attacks are *completely undetectable* and thus avoid all detection mechanisms [12]. Hence, the main idea of the countermeasure studied here is to forgo detection and concentrate on actively removing the covert channel that the subverted implementation aims to open, thereby preventing the leakage of secret information.

4.1 Fundamental Properties of Reverse Firewalls

The firewall itself does not have any secrets and should not gain information about any secret of its main party. Therefore, it can speak of the correctness of a message no more than an eavesdropper. However, the firewall can easily do a syntactical check and only let through messages that have the correct syntax. For example, it can ignore messages consisting of two group elements, if the protocol only expects one group element to be sent.

In [46, 33], the authors not only introduced firewalls as a concept, but also defined different properties that are desirable for such a firewall to have. Informally, we require our firewalls to have the following properties:

Functionality Maintenance: If the implementation is not subverted, the honest firewall should not break the functionality of the underlying protocol.

Security Preservation: If a subverted party is used with an honest firewall, all security guarantees of the underlying protocol should be preserved.

Exfiltration Resistance: The subverted implementation should not be able to leak sensitive information through an honest firewall. This is captured by comparing a subverted party together with an honest firewall against an honest party together with an honest firewall.

The authors of [46, 33] further divide the last two definitions into a strong and a weak version depending on whether this holds against *any* PPT adversary or only against all PPT adversaries that maintain functionality. Such “functionality-maintaining adversaries” correspond to the subversions considered by us. In this paper, we require the firewalls to only fulfil the so-called *weak* properties.

In [46, 33], their firewalls also guarantee the following additional properties:

Stackability: A party should be allowed to have arbitrarily many firewalls. A single correct firewall should already guarantee security.

Transparency: The other parties do not need to be aware of the existence of the firewall(s). This is captured by comparing an honest party together with an honest firewall against an honest party without firewall.

In this work, we will call their [46, 33] notion of transparency *outer transparency*, as it only concerns the other parties, i.e., the view from the outside. Furthermore, we will strengthen the notion of stackability to the notion of *inner transparency*, where we require that an honest implementation needs to behave the same whether a firewall is present or not. We will show that a firewall having both outer and inner transparency (which we call *strong transparency*) allows to keep (nearly) all security guarantees of the underlying protocol.

4.2 Modelling Properties of Reverse Firewalls in UC

While the *functionality maintaining*, *security preserving*, and *exfiltration resisting* properties explained above are clearly needed for reverse firewalls to work at all, there are other useful properties worth to consider. In the following, we will formally define the notions of *transparency* and *anti-signalling* that capture (or extend) the original properties of Mironov and Stephens-Davidowitz [46] in the UC-setting. Furthermore, as we will see, these properties also allow to simplify the security analysis significantly, as we do not need to “reprove” the UC-simulatability. In the following, Π denotes the run of an incorruptible⁶ party $\hat{\mathcal{P}}$ and Π' is the *same* protocol, but run by the composed party $\hat{RF} \circ \hat{\mathcal{P}}$.

Transparency As mentioned above, we extend the definition of *transparency* by [46] and coin the terms *strong transparency* and *outer transparency*. Roughly speaking, the latter is given if an honest party without a firewall is indistinguishable from an honest party together with the firewall behaving honestly, while strong transparency is given if, additionally, the party itself does not have to change its behaviour when using such a firewall.

Definition 5. (*Transparency of Firewalls*) Let $\mathcal{P}$ be a party, RF its firewall, and $\hat{\mathcal{P}}$ (resp. $\hat{RF}$) be the incorruptible versions of $\mathcal{P}$ (resp. RF). Furthermore, let Π be the protocol run by $\hat{\mathcal{P}}$ and Π' be the protocol run by the composed party $\hat{RF} \circ \hat{\mathcal{P}}$. The firewall RF provides *outer transparency* if for all PPT environments $\mathcal{Z}$ it holds that $|\Pr[\text{EXEC}_{\Pi, \mathcal{A}, \mathcal{Z}}(\kappa, r) = 1] - \Pr[\text{EXEC}_{\Pi', \mathcal{A}, \mathcal{Z}}(\kappa, r) = 1]|$ is negligible, where $\mathcal{A}$ is the dummy adversary. If, additionally, the protocols Π and Π' contain the same code π for their respective party $\hat{\mathcal{P}}$, then RF provides *strong transparency*.

Strong transparency means that the behaviour does not change regardless of whether $\hat{\mathcal{P}}$ or $\hat{RF} \circ \hat{\mathcal{P}}$ is used. This includes that there is no additional communication to/from the party, so a party does not need to “know” how many firewalls are used or provide different interfaces for different firewalls. Note that this also extends to the behaviour of the firewall, as the behaviour of a composed party $\hat{RF} \circ \hat{\mathcal{P}}$ should not change when employing it together with an additional firewall as $\hat{RF}' \circ (\hat{RF} \circ \hat{\mathcal{P}})$. This directly implies functionality maintenance.

Note that we will only use strong transparency in this work and the notion of outer transparency is mainly introduced to highlight the difference to previous works. Outer transparency can allow for a feedback channel between the firewall and the party, which can weaken the goal of the initial property of strong transparency. For further discussion, see Appendix A.

Non-Signalling Composed Party Another property to model is the notion that the firewall counteracts the leakage of a subverted party by making the combination of party and firewall *non-signalling*. The goal is that not even

⁶ I.e., a party ignoring the corruption messages send by the adversary.

the adversary can distinguish the behaviour of $RF \circ \bar{\mathcal{P}}$ from $RF \circ \mathcal{P}$. We call such a firewall *anti-signalling* (similar to *exfiltration-resistance* by Mironov and Stephens-Davidowitz [46] and *strong sanitation* by Chakraborty et al. [26]).

Definition 6. (*Anti-Signalling Firewalls*) Let $\mathcal{P}$ be a party and RF its firewall. Let $\bar{\mathcal{P}}$ be the subvertable version of $\mathcal{P}$. Let $\hat{\mathcal{P}}$ and $\hat{RF}$ the incorruptible version of $\mathcal{P}$ and RF , respectively. Let Π be the protocol run by the composed party $\hat{RF} \circ \bar{\mathcal{P}}$. Let Π' be the protocol run by $\hat{\mathcal{P}}$ and $\hat{RF}$. The firewall RF is anti-signalling if for all PPT environments $\mathcal{Z}$ it holds that $|\Pr[\text{EXEC}_{\Pi, \mathcal{A}, \mathcal{Z}}(\kappa, r) = 1] - \Pr[\text{EXEC}_{\Pi', \mathcal{A}, \mathcal{Z}}(\kappa, r) = 1]|$ is negligible, where $\mathcal{A}$ is the dummy adversary.

In most applications, there are *perfectly* anti-signalling firewalls, i.e., the runs $\text{EXEC}_{\Pi, \mathcal{A}, \mathcal{Z}}(\kappa, r)$ and $\text{EXEC}_{\Pi', \mathcal{A}, \mathcal{Z}}(\kappa, r)$ are distributed identically. The definition of anti-signalling is visualised in Fig. 3.

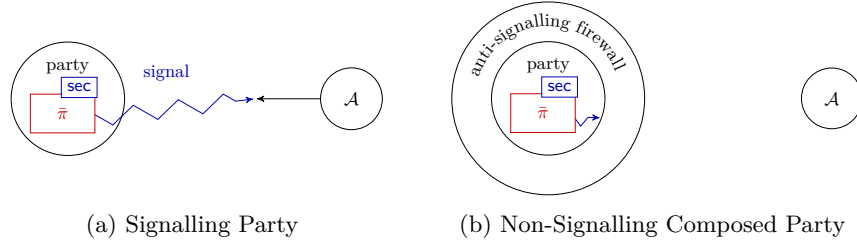


Fig. 3: Informal visualisation of a signalling party (a), which together with an anti-signalling firewall can be turned into a non-signalling composed party (b).

Now, if we are able to construct an anti-signalling firewall providing strong transparency, the interaction of a subverted party using an honest firewall with another party is indistinguishable from the interaction of honest parties. The notion of anti-signalling allows to replace the subverted party using the honest firewall with an honest party using the honest firewall and the strong transparency allows to replace the honest party using the honest firewall by an honest party.

5 Security Analysis of Subversion Corrupted Protocols with Reverse Firewalls

In this section, we want to show how one analyses the security of a protocol susceptible to subversion corruptions that is at the same time protected by firewalls. Therefore, we will utilise specific properties of firewalls discussed in Section 4 to give a generic security proof. We will focus on the adaptation of existing UC-secure protocols, where, similar to all other previous approaches, a combined party containing a malicious firewall will be treated as malicious. In Section 8, we show another approach, where we need to adapt the protocols, but can also guarantee security against corrupted firewalls.

Subversion corruptions are an attack vector to real-world protocols. However, we can already get rid of this attack vector in a hybrid step following the corruption behaviour of the transition table in Table 2 in the real world, such that we do not need to forward subversion corruptions to the ideal world. This gives us the advantage that we can easily make use of given standard UC functionalities without reproving elementary theorems of plain UC. Examples for basic relevant protocols, namely oblivious transfer, will be provided in the next section.

5.1 How to Analyse Subversion Resilience of a Protocol

In general, we want that a protocol $\Pi \geq_{\text{UC}} \mathcal{F}$ under subversion corruption. Therefore, our main focus lies in proving that a protocol Π already UC-emulates a classic protocol Π' under subversion corruption in the real world. By showing that subversion corruptions are *not* necessary in the ideal world and keeping the transitivity of UC-emulation in mind, we get that Π' , and thereby Π , UC-realises an ideal functionality $\mathcal{F}$ under subversion corruption. For a more in-depth explanation, we go through the following steps:

First. We assume that there exists a classic protocol Π' ((b) in Fig. 4) which will be the base protocol of Π ((a) in Fig. 4), meaning that no subversion corruptions (and no firewalls) are considered. In particular, Π' UC-realises the desired ideal functionality $\mathcal{F}$ ((c) in Fig. 4) under (semi-honest and) malicious corruption.

Second. To begin with, we give every main party an interface to receive a secret sec from the environment. Note that in a real computer system, the secret could also already be in the state of the party. By giving $\mathcal{Z}$ the power to set the secret, we model the adversary to be even more powerful. For the security analysis, the **secret** interface has to be added to every protocol, i.e., even to the base protocol and the ideal functionality. This has no influence on the security behaviour apart from the modelling aspect of subversion resilience. Not only does this adaptation explicitly model the subversion goal of leaking a secret, it also is necessary to be able to simulate the case of a corrupted firewall, which in turn is vital to analyse the advantages and disadvantages of such an additional protection layer.

Third. We adapt Π' such that we can protect it against subversion attacks. All adaptations follow the model of Section 3, i.e., subversion corruptions and sub-party corruptions of the firewall are possible. This allows us to analyse its security

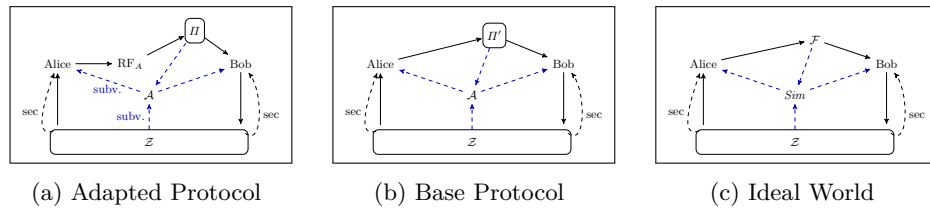


Fig. 4: The adapted protocol (a) of the base protocol (b) models subversion corruption and firewalls. Protocol (b) UC-realises the ideal functionality in (c).

under subversion corruption. Additionally, following the modelling of Section 3.2, we give a firewall RF as a sub-party to every main party $\mathcal{P}$ which could leak something about its secret via subversion during the protocol run. The firewall should be strongly transparent and anti-signalling, a combination of properties which, as we will show, allows to protect against subversion corruptions.

These protocol adaptations yield the protocol Π ((a) in Fig. 4).

Fourth. We show that the adapted protocol Π and the base protocol Π' are indistinguishable, i.e., we give a simulator $Sim_{\Pi'}$ such that for all PPT environments $\mathcal{Z}$ it holds that $\Pr[\text{EXEC}_{\Pi, \mathcal{A}, \mathcal{Z}} = 1] - \Pr[\text{EXEC}_{\Pi', Sim_{\Pi'}, \mathcal{Z}} = 1]$ is negligible. We show that $\Pi \geq_{UC} \Pi'$ under party-wise subversion, semi-honest, and malicious corruption following the corruption behaviour transition given in Table 2.

Fifth. From the transitivity of UC-emulation [17, Sec. 4.3] it follows that if $\Pi \geq_{UC} \Pi'$ and $\Pi' \geq_{UC} \mathcal{F}$, then $\Pi \geq_{UC} \mathcal{F}$.

Formally speaking, we want to prove our following main theorem, which states that the adapted protocol Π is indistinguishable from the base protocol Π' . This indicates that both protocols are resilient against subversion corruptions via the help of reverse firewalls.

Theorem 1 (Main Theorem). *Let Π be the protocol with main parties $\mathcal{P}_i$ under party-wise subversion, semi-honest, and malicious corruption, and the (sub-) parties $RF_{\mathcal{P}_i}$ under party-wise semi-honest and malicious corruption in the $\{\mathcal{F}_{CRS}, \mathcal{F}_{AUTH}\}$ -hybrid model. Let Π' be the protocol with main parties $\mathcal{P}'_i$ under party-wise semi-honest and malicious corruption in the $\{\mathcal{F}_{CRS}, \mathcal{F}_{AUTH}\}$ -hybrid model. Let the corruption type of the parties $\mathcal{P}_i$ and $RF_{\mathcal{P}_i}$ transition from their individual party-wise corruption to their composed classic corruption behaviour type following the corruption transition Table 2. Let $RF_{\mathcal{P}_i}$ be strongly transparent and anti-signalling. Let r be the randomness and κ be the security parameter. There exists a simulator $Sim_{\Pi'}$ such that for all PPT environments $\mathcal{Z}$, we have $\text{EXEC}_{\Pi, \mathcal{A}, \mathcal{Z}}(\kappa, r) \approx \text{EXEC}_{\Pi', Sim_{\Pi'}, \mathcal{Z}}(\kappa, r)$, where $\mathcal{A}$ is the dummy adversary.*

Proof Idea of Theorem 1. When proving Theorem 1, we need to show that $\text{EXEC}_{\Pi, \mathcal{A}, \mathcal{Z}}(\kappa, r) \approx \text{EXEC}_{\Pi', Sim_{\Pi'}, \mathcal{Z}}(\kappa, r)$, where $\text{EXEC}_{\Pi, \mathcal{A}, \mathcal{Z}}(\kappa, r)$ is the execution of the adapted protocol ((a) in Fig. 4) and $\text{EXEC}_{\Pi', Sim_{\Pi'}, \mathcal{Z}}(\kappa, r)$ is the execution of the base protocol ((b) in Fig. 4). In the following, we give the proof idea for the most interesting cases. For the case of an honest main party $\mathcal{P}$ and an honest firewall RF in Π , we can show that this is indistinguishable from an honest party $\mathcal{P}$ in Π' if RF fulfils the notion of strong transparency (Definition 5). The case where the RF is semi-honest works similarly as an honest $\mathcal{P}$ does not leak any secret. Hence, it translates to an honest $\mathcal{P}$ in Π' . For the case of a subverted main party $\mathcal{P}$ and an honest firewall RF in Π , we can show that this is indistinguishable from an honest party $\mathcal{P}$ in Π' if RF fulfils the notion of anti-signalling (Definition 6). The firewall RF cannot offer protection in the case of a maliciously corrupted main party $\mathcal{P}$, thus, this directly translates to the maliciously corrupted case without RF . For the case of a malicious firewall RF in Π , it can impersonate the honest or subverted main party $\mathcal{P}$. Thus, we need to treat the composed party as malicious. For countermeasures against this, we

refer to Section 8 for a discussion of how one can translate an honest $\mathcal{P}$ with malicious RF in Π to an honest party in Π' . The case of a subverted $\mathcal{P}$ and a semi-honest RF in Π is indistinguishable from a semi-honest $\mathcal{P}$ in Π' mainly because in Π' , the simulator learns the secret of the semi-honestly corrupted party as it is part of the party's state.

A more extensive description of each of these cases is given below, where we provide an argument for every line in Table 2, i.e., all possible cases given the static corruption combinations. For the most interesting cases of subversion corrupted parties, we will provide a simulator $Sim_{\Pi'}$ below, which also provides insights into the communication flow. There, we also give further discussions of the corruption handling, including the steps required to get to the ideal world. For completeness, there, we also provide the code for the shell of a party in the base protocol Π' . Note that its main difference to a general shell of party $\mathcal{P}$ in a standard protocol is the interface to insert a secret. With the shell code behaviour of $\mathcal{P}$ and RF in the adapted protocol Π , one can see the indistinguishability regarding the corruption handling via the Corruption Aggregation ITI. Additionally, one can see that the transitivity of UC-emulation from the plain UC framework is directly applicable. Further details on the transitivity of UC-emulation are discussed below. Finally, we explain the steps to achieve subversion-resilience in our model in Section 5.3.

To prove that Π UC-emulates Π' , we have to show that it is indistinguishable for $\mathcal{Z}$ which protocol is executed. $Sim_{\Pi'}$ simulates the (corruption) behaviour of the main party $\mathcal{P}$ and its sub-party RF in Π to be indistinguishable from the (corruption) behaviour given by Table 2 of the main party $\mathcal{P}$ in Π' . The simulator only captures the communication flow till and from the network as after that, it works like the dummy adversary in Π' . We assume that the network is captured by $\mathcal{F}_{AUTH}$ as we analyse protocols in the $\{\mathcal{F}_{CRS}, \mathcal{F}_{AUTH}\}$ -hybrid model. Note that it does not matter how the other party (or parties) are corrupted in the protocol and whether they have firewalls installed, because the simulator can be used party-wise and combined modularly. We only consider static corruption and provide a simulator consisting of a concatenation of all case-wise simulators. Hence, we can focus on the two-party case with the view on one main party $\mathcal{P}$ and its sub-party RF which can be extended modularly to more parties. We make use of $Sim_{\Pi'}$ for all cases considering a subverted party.

$Sim_{\Pi'}$

For a protocol Π' , $Sim_{\Pi'}$ simulates the protocol Π with main party $\mathcal{P}$ under static subversion, semi-honest, or malicious corruption, and its sub-party RF under static semi-honest or malicious corruption. A “network” is used to represent the in- and output interface of the (composed) party, i.e., $\mathcal{F}_{AUTH}$, and all further protocol parts.

Behaviour:

\\ Case: Corruption Handling

- Upon receiving (**request-corruption**, *sid*) from party $\mathcal{P}$,

```

send (request-corruption, sid) to  $\mathcal{Z}$ .
- Upon receiving (corrupt, sid, cp,  $\mathcal{P}$ ) from  $\mathcal{Z}$ ,
  parse cp to store  $cp(\mathcal{P})$  and  $cp(RF)$ ,
  if  $cp(\mathcal{P}) == (\text{subversion} : \bar{\pi})$ :
    store (sid,  $\mathcal{P}$ , subversion,  $\bar{\pi}$ ),
    determine  $cp(RF \circ \mathcal{P})$  from the transition Table 2,
    send (corrupt, sid,  $cp(RF \circ \mathcal{P})$ ) to  $\mathcal{P}$ ,
    and ignore further corrupt messages.
  \\ Case: Subverted party  $\mathcal{P}$  and honest firewall  $RF$ :
- On all other backdoor messages, forward them to the respective recipient. \\
  Since firewall honest, there are no backdoor messages between firewall and  $\mathcal{Z}$ .
  \\ Case: Subverted party  $\mathcal{P}$  and semi-honest firewall  $RF$ :
- Upon receiving backdoor messages (semi-honest, sid, state) from  $\mathcal{P}$ , simulate
  the subversion corruption with  $\bar{\pi}$  and the firewall on state in-the-head, and
  output the updated state (semi-honest, sid, state') in the name of  $RF$  to  $\mathcal{Z}$ .
- On "continue" from  $\mathcal{Z}$  to  $RF$ , send "continue" to  $\mathcal{P}$ .
- On all other backdoor messages, forward them to the respective recipient.
  \\ Case: Subverted party  $\mathcal{P}$  and malicious firewall  $RF$ :
- On input from  $\mathcal{Z}$  to  $\mathcal{P}$ ,
  run in-the-head  $\mathcal{P}$ 's  $\bar{\pi}$  and send the output in the name of  $RF$  to  $\mathcal{Z}$ .
- On input in the name of  $RF$  to the network, forward it to the network as if  $\mathcal{P}$ 
  would be malicious.
- On output from the network to  $\mathcal{P}$ , forward it as input of  $RF$  to  $\mathcal{Z}$ .
- On output in the name of  $RF$  to  $\mathcal{P}$ , run in-the-head  $\mathcal{P}$ 's  $\bar{\pi}$  and send the output
   $\bar{\pi}$  would send in the name of  $\mathcal{P}$ .

```

5.2 On Corruption Handling

In the adapted protocol Π , the shell of $\mathcal{P}$ receives the corruption parameter $cp(\mathcal{P})$ as well as $cp(RF)$. The shell corrupts the body of $\mathcal{P}$ according to $cp(\mathcal{P})$ and informs its firewall by invocation about its corruption parameter $cp(RF)$. The Corruption Aggregation ITI will be informed by $\mathcal{P}$'s shell about the composed corruption behaviour $cp(RF \circ \mathcal{P})$ according to Table 2.

The simulator $Sim_{\Pi'}$ corrupts $\mathcal{P}'$ in the base protocol Π' directly with the composed corruption behaviour $cp(RF \circ \mathcal{P})$ according to Table 2, which then will notify the Aggregation ITI. As in both cases, the Corruption Aggregation ITI will never be informed by or about a firewall and the corruption parameters requested by $\mathcal{Z}$ from the Aggregation ITI are identical, we can conclude that the corruption handling is perfectly indistinguishable for the environment, i.e., $EXEC_{\Pi, \mathcal{A}, \mathcal{Z}}(\kappa, r) \approx EXEC_{\Pi', Sim, \mathcal{Z}}(\kappa, r)$.

The shell of party $\mathcal{P}$ in the base protocol Π' has to be adapted regarding the corruption handling in such a way that it only reads the respective corruption parameter $cp(\mathcal{P})$, which is something one can expect any shell of a general party to do. Additionally, it has to be able to handle the input of a secret as described earlier.

Code of shell of party $\mathcal{P}$ in the base protocol Π'

- On **(secret, sid, sec $\mathcal{P}$)** from $\mathcal{A}$, write **sec $\mathcal{P}$** on *secret tape*.
 $\backslash\backslash$ Which is just one part on the tape of the internal state.
- Upon invocation, send **(request-corruption, sid)** to $\mathcal{A}$.
- On next activation; on **(corrupt, sid, cp)** from $\mathcal{A}$:
 Parse cp to get $cp(\mathcal{P})$ and ignore further part of cp string.
 If $cp(\mathcal{P}) \notin \{\text{honest}, \text{semi-honest}, \text{malicious}\}$
 set $cp(\mathcal{P}) := \text{honest}$.
 Send **(corrupt, sid, cp($\mathcal{P}$), $\mathcal{P}$)** to Agg-ITI.
- On activation return by Agg-ITI, proceed according to the respective specific corruption type described, e.g., in [17, Sec. 7.1.1].
- Ignore further corruption messages.

To handle the semi-honest corruption of the RF while $\mathcal{P}$ is subverted in Π , which will be treated as a semi-honest corruption of $\mathcal{P}$ in Π' , any ideal functionality has to give the **sec** to the simulator.

Honest $\mathcal{P}$ and honest RF behave indistinguishable from honest $\mathcal{P}_{\Pi'}$. If RF is strongly transparent, $\text{EXEC}_{\Pi, \mathcal{A}, \mathcal{Z}}(\kappa, r) \approx \text{EXEC}_{\Pi', \text{Sim}, \mathcal{Z}}(\kappa, r)$, where both the main party in Π and its RF is honest, and the main party in Π' is honest as well. Hence, it has no influence on the simulation. The distributions are identically on both sides. Therefore, the simulator can just ignore the firewall as no secret will be leaked.

Honest $\mathcal{P}$ and semi-honest RF behave indistinguishable from honest $\mathcal{P}_{\Pi'}$. From the analysis above, we know that an honest party together with an honest, strongly-transparent firewall is indistinguishable from honest $\mathcal{P}_{\Pi'}$. When now replacing the firewall with the semi-honestly corrupted RF , the composed party *behaves* the same as with the honest firewall. The only difference between the composed parties is that now, the adversary additionally learns about its state. However, since the party is honest, no secret will be leaked to the firewall. Consequently, we can see the composed party as honest. The simulator has to generate the state and output of RF by running the code on the output of $\mathcal{P}$, which the simulator simulates during the network communication part.

Honest $\mathcal{P}$ and malicious RF behave indistinguishable from honest/malicious $\mathcal{P}_{\Pi'}$. This case is an interesting edge-case as the firewall is maliciously corrupted but the party itself is honest. Because of the latter, no secret will be leaked by the party, which in turn means that the firewall can leak nothing. However, the firewall communicates in the name of $\mathcal{P}$ and, therefore, can impersonate it by generating its own messages. There are different approaches to handle this situation: The worst-case consideration would be viewing the composed party as maliciously corrupted. In this case, the simulator could simulate the behaviour of honest $\mathcal{P}$ and malicious RF . Remark that Canetti models malicious corruptions in a way that “in an activation due to an incoming input or subroutine-output, the shell

sends the entire local state to the adversary” [17, §7.1.1 Byzantine corruption, line 6f]. This modelling also captures what one would expect by maliciously corrupting a party —the input to this party goes to the adversary. Therefore, Sim receives the input from $\mathcal{Z}$, which gives Sim the power to run $\mathcal{P}$ ’s code in the head on this input to simulate an honest output for the RF . On message from RF to $\mathcal{P}$, Sim runs $\mathcal{P}$ in the head. This completes the simulation and both sides are indistinguishable. However, malicious corruption behaviour would imply that none of the security properties is guaranteed any more, even though this is generally not the case. To get the composed party to behave like an honest party, one has to circumvent impersonation attacks of RF . This is captured with modelling and more practical solutions as discussed below in Section 8.

Malicious $\mathcal{P}$ and honest/semi-honest/malicious RF behave indistinguishable from malicious $\mathcal{P}_{II'}$. The two executions are indistinguishable as Sim gets all in-/output to/from the malicious $\mathcal{P}$ and can run RF ’s code in-the-head before sending it to $\mathcal{Z}$ or the network, respectively. In the case where RF is semi-honest, output the produced state of RF to $\mathcal{Z}$ and continue on "ok" from $\mathcal{Z}$. In the case where RF is malicious, simulate communication between $\mathcal{P}$ and RF as between two malicious parties, i.e., send each in-/output to $\mathcal{Z}$.

Remark 5. Following Section 2.3.3 in [37], we know that the UC-simulation under malicious corruption implies the UC-simulation under augmented semi-honest corruption. This means that if we can show that the base protocol is secure if $\mathcal{P}_{II'}$ is malicious, it is also secure if $\mathcal{P}_{II'}$ is augmented semi-honest. Again, if the base protocol II' is also secure against semi-honest corruptions, this type of corruption can also be considered for the composed parties.

Subverted $\mathcal{P}$ and honest RF behave indistinguishable from honest $\mathcal{P}_{II'}$. This is the most relevant case of this work. Because RF satisfies the definition of anti-signalling (Definition 6), it holds that $EXEC_{II, \mathcal{A}, \mathcal{Z}}(\kappa, r) \approx EXEC_{II', Sim_{II'}, \mathcal{Z}}(\kappa, r)$, where the main party in II is subversion corrupted and its RF honest, and the main party in II' is honest. In this case, the composed party $RF \circ \mathcal{P}$ is non-signalling and, hence, subversion resilient. This means that the party cannot leak the secret via subversion of the protocol.

Subverted $\mathcal{P}$ and semi-honest RF behave indistinguishable from semi-honest $\mathcal{P}_{II'}$. The two executions are indistinguishable as $Sim_{II'}$ can produce the state of the semi-honestly corrupted RF by running its code in the head after applying $\bar{\pi}$ to $\mathcal{P}_{II'}$ ’s state, i.e., replacing the randomness function. Since a semi-honest firewall still follows its code, any leakage output through the subverted $\mathcal{P}$ will be removed, but the secret sec would be leaked via the state. Note that while the executions are indistinguishable, we can simulate this case, though the protocol is not subversion resilient any more.

Subverted $\mathcal{P}$ and malicious RF behave indistinguishable from malicious $\mathcal{P}_{II'}$. This case works similar to the case of an honest $\mathcal{P}$ and a malicious RF . $Sim_{II'}$ also receives the secret as an input to $\mathcal{P}$ and can, therefore, simulate. However, the secret is leaked as well. As $Sim_{II'}$ knows the in-/output of $\mathcal{P}$ and RF , it can easily simulate running $\bar{\pi}$ and RF in-the-head if needed. Hence, both sides

are indistinguishable. One cannot hope to achieve an honest behaviour for the composed party as the secret is leaked by the malicious firewall, but one can reduce the attack vector of impersonation attacks of RF , as discussed in Section 8.

Remark 6. If the party is subverted, we can view the composed party of a subverted party and its corrupted firewall as a corrupted party. This might seem like an exaggeration, however, in the worst case, a subversion could leak the seed of the whole state of $\mathcal{P}$ which would then be equal to the direct semi-honest corruption of $\mathcal{P}$. Hence, the composed party of a subverted $\mathcal{P}$ with a corrupted RF behaves, in the worst case, the same as a corrupted $\mathcal{P}$. However, we can show that a base protocol to which firewalls have been added still emulates the ideal functionality of the base protocol. In the case of an honest party and a malicious firewall, one needs to take care of impersonation attacks by the firewall (see Section 8). But if we add protection against such impersonation attacks of the firewall, we can treat the behaviour of a subverted party and its malicious firewall like the corruption behaviour of a semi-honest party.

Remark 7. We do not restrict the environment in any of the UC-emulations. We still allow $\mathcal{Z}$ to send subversion corruption messages, but they are ignored by the shells. This emphasises the fact that we protect a party attacked by subversion corruptions already in the real world by implementing the RF there. So it is rather the shell in the adapted real world protocol which acts differently, however, this has to take place anyway by adding both subversion attacks and firewalls.

As we do not adapt the UC framework, i.e., all results are in the plain UC model, we can make use of the transitivity of UC-emulation. The subversion resilience of the ideal functionality is given as the subversion corruption is not necessary, i.e., ignored, in the ideal world. This means that the secret cannot be leaked by any party. The transitivity from the subversion resilient real world protocol to the ideal functionality implies this fact.

Corollary 1. *Given Theorem 1, it follows that $\Pi \geq_{UC} \mathcal{F}$.*

From the transitivity of UC-emulation [17] and Theorem 1, it follows directly that $\Pi \geq_{UC} \mathcal{F}$ because $\Pi \geq_{UC} \Pi'$ and $\Pi' \geq_{UC} \mathcal{F}$.

5.3 Steps for Applying Our Model

This list is part of our prior steps describing how to prove the security, but here we specify the steps one has to fulfil based on Theorem 1.

Let Π' be a base protocol securely UC-realising an ideal functionality $\mathcal{F}$ in the presence of semi-honest and malicious corruptions. To analyse the resilience against subversions via firewalls, one only has to proceed with the following steps:

1. Give every main party in every protocol an explicit **sec** interface as described in Section 3.1.
2. Adapt the protocol according to Section 3 to both expand the corruption vector by subversion and corruptions of possible firewalls, and add firewalls as a protection layer to the protocol.

3. Show that the firewalls are both strongly transparent (Definition 5) and anti-signalling (Definition 6), and that the new protocol Π still fulfils correctness.

Following Theorem 1, the new protocol Π UC-emulates the base protocol Π' where the main parties are honest, semi-honest, malicious, or subversion corrupted with a subversion code $\bar{\pi}$, and the respective sub-parties RF are honest, semi-honest, or malicious. With the indistinguishability of the adapted protocol with respect to a base protocol, one can see that the protocol is subversion resilient via strongly transparent and anti-signalling firewalls in the real world. Following Corollary 1, when Π' also UC-realises the ideal functionality $\mathcal{F}$ in the $\{\mathcal{F}_{\text{CRS}}, \mathcal{F}_{\text{AUTH}}\}$ -hybrid model, one gets that Π UC-realises $\mathcal{F}$ while fulfilling subversion resilience.

Remark 8. Even though Theorem 1 is applicable for maliciously corrupted firewalls, as discussed in Section 8, protecting against impersonation attacks requires additional measures.

6 Commitments

The considered commitment scheme has been brought up as a running example throughout this paper. It is based on the one in [21] and is a transformation of the oblivious transfer scheme by the same authors presented below in Section 7. Additionally, as they also base their commitment scheme on the one in [21], the bit version of the commitment discussed in [26] is equal to ours when setting the input of their firewall to zero. As was done in [26], our protocol is in a modified common reference string (CRS) model where the first three elements are assumed to be random. We directly make use of the security analysis of [21] and do not need to “reprove” the UC-security of the protocol.

Given a $\Pi'_{\text{COM}} \geq_{\text{UC}} \mathcal{F}_{\text{COM}}$ in the $\{\mathcal{F}_{\text{CRS}}, \mathcal{F}_{\text{AUTH}}\}$ -hybrid model under malicious corruption, we model Π_{COM} between two parties. Note that since this is a non-interactive commitment, only the committer needs a firewall which rerandomises the commitment $c' = c \cdot \text{COM}(0, r \xleftarrow{\$} \mathbb{Z}_p)$ and later opens to the message with the rerandomised decommitment. The environment gets the additional option to subversion-corrupt the committer with a subverted code, i.e., an adversarially-chosen randomness function.

6.1 Ideal Functionality: Commitment

The ideal functionality of the commitment is the following:

$\mathcal{F}_{\text{COM}}$
Two-party ideal bit commitment protocol between a committer $\mathcal{C}$ and a verifier $\mathcal{V}$.

- Upon receiving an input $(\text{commit}, \text{sid}, v)$ from $\mathcal{C}$, where $v \in \{0, 1\}$, record the tuple (sid, v) and generate a public delayed output $(\text{committed}, \text{sid})$ to $\mathcal{V}$. Ignore subsequent commands of the form $(\text{commit}, \text{sid}, \cdot)$.
- Upon receiving an input $(\text{open}, \text{sid})$ from $\mathcal{C}$, generate a public delayed output $(\text{opened}, \text{sid}, v)$ to $\mathcal{V}$. Ignore subsequent commands of the form $(\text{open}, \text{sid}, \cdot)$.
- \\ Corruption Handling:
 - On input $(\text{secret}, \text{sid}, \mathcal{P}, \text{sec}_{\mathcal{P}})$, store $(\text{secret}, \text{sid}, \mathcal{P}, \text{sec}_{\mathcal{P}})$. If $\mathcal{P}$ is semi-honestly or maliciously corrupted, send $(\text{secret}, \text{sid}, \mathcal{P}, \text{sec}_{\mathcal{P}})$ to Sim .
 - Ignore all further inputs.

6.2 Real Protocol: Subversion-Resilient Commitment Based on DLog [21]

The group $\mathbb{G}$ with generator g and the field $\mathbb{Z}_q$ are public inputs. The adapted protocol including the firewalls is the following:

$$\Pi_{\text{srCOM}}^{\mathcal{F}_{\text{CRS}}, \mathcal{F}_{\text{AUTH}}}$$

Realises:

Subversion-resilient two-party bit commitment between a main party committer $\mathcal{C}$, its sub-party firewall $RF_{\mathcal{C}}$, a main party receiver $\mathcal{R}$, and an adversary $\mathcal{A}$.

Parameters:

- Functionality $\mathcal{F}_{\text{AUTH}}$.
- Functionality $\mathcal{F}_{\text{CRS}}$:
 - On sending $(\text{value}, \text{sid})$ to $\mathcal{F}_{\text{CRS}}$, every party receives $(\text{crs}, \text{sid}, (g, T_1, h, T_2))$.
 - Static subversion corruption of $\mathcal{C}$ with honest $RF_{\mathcal{C}}$ is pictured, others omitted.
 - Every main-party has an empty initialised secret tape.

Behaviour of Party $\mathcal{C}$:

- \\ Secret Handling:
 - Upon receiving $(\text{secret}, \text{sid}, \text{sec})$ from $\mathcal{Z}$, write sec on the secret tape.
- \\ Commitment Handling:
 - Upon receiving $(\text{commit}, \text{sid}, v)$ from $\mathcal{Z}$, where $v \in \{0, 1\}$, get d from the randomness function, \\ presumably subverted and set $c := (c_0, c_1) = \text{Com}(v, d) = (g^d \cdot T_1^v, h^d \cdot T_2^v)$. Send $(\text{send}, \text{sid}, \mathcal{C}, \mathcal{V}, c)$ to $RF_{\mathcal{C}}$ via an immediate channel.
 - Upon receiving $(\text{open}, \text{sid})$ from $\mathcal{Z}$, output $(\text{send}, \text{sid}, \mathcal{C}, \mathcal{V}, (v, d))$ to $RF_{\mathcal{C}}$.

Behaviour of Sub-party $RF_{\mathcal{C}}$:

- Upon receiving $(\text{send}, \text{sid}, \mathcal{C}, \mathcal{V}, c)$ from $\mathcal{C}$, sample $r \xleftarrow{\$} \mathbb{Z}_q$, and set $c' := (c'_0, c'_1) = c \cdot \text{Com}(0, r) = (c_0 \cdot g^r, c_1 \cdot h^r)$. Send $(\text{send}, \text{sid}, \mathcal{C}, \mathcal{V}, c')$ in the name of $\mathcal{C}$ to $\mathcal{F}_{\text{AUTH}}$.
- Upon receiving $(\text{send}, \text{sid}, \mathcal{C}, \mathcal{V}, (v, d))$ from $\mathcal{C}$, set $d' := d + r$, and send $(\text{send}, \text{sid}, \mathcal{C}, \mathcal{V}, (v, d'))$ to $\mathcal{F}_{\text{AUTH}}$.

Behaviour of Party $\mathcal{V}$:// Secret Handling same as for Party $\mathcal{C}$.

// Commitment Handling:

- Upon receiving $(\text{sent}, \text{sid}, \mathcal{C}, \mathcal{V}, c')$ from $\mathcal{F}_{\text{AUTH}}$, output $(\text{committed}, \text{sid})$ to $\mathcal{Z}$.
- Upon receiving $(\text{sent}, \text{sid}, \mathcal{C}, \mathcal{V}, (v, d'))$ from $\mathcal{F}_{\text{AUTH}}$, output $(\text{opened}, \text{sid}, v)$ to $\mathcal{Z}$.

The real run of the commitment scheme is illustrated in Fig. 5.

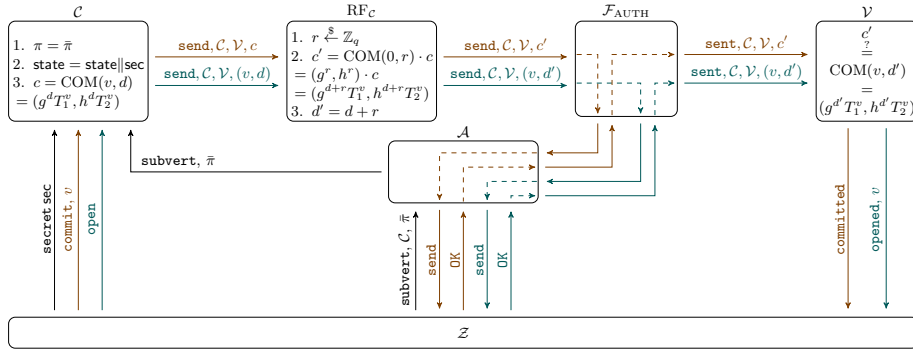


Fig. 5: Real run of the commitment scheme (pictured without $\mathcal{F}_{\text{CRS}}$).

6.3 Security Proof

This protocol can be shown to fulfil the following:

Theorem 2. *The protocol Π_{srCOM} UC-realises the functionality $\mathcal{F}_{\text{COM}}$ in the $\{\mathcal{F}_{\text{CRS}}, \mathcal{F}_{\text{AUTH}}\}$ -hybrid model, where the main parties are maliciously, semi-honestly or subversion corrupted with a subversion code $\bar{\pi}$, and the respective sub-parties RF are honest, semi-honest or malicious.*

The base commitment scheme in [21] proves security under malicious corruption. Again, this implies security under augmented semi-honest corruption. Plain semi-honest security, however, has not been shown yet. We leave this as an open problem to the reader; following Theorem 1, this would first require proving the case of the subverted party and plain semi-honestly corrupted firewall behaving indistinguishable from a plain semi-honest party.

Proof. The proof follows the steps described in Section 5.3. From Corollary 1, we know that if Π_{srCOM} UC-emulates Π_{COM} and Π_{COM} UC-realises $\mathcal{F}_{\text{COM}}$, then Π_{srCOM} UC-realises $\mathcal{F}_{\text{COM}}$. In [21], it was proved that a variant of Π_{COM} without T_2 in the CRS UC-realises $\mathcal{F}_{\text{COM}}$. For our proof, we, hence, only have to focus

on the former part of Π_{srCOM} UC-emulating Π_{COM} . Here, we have to show that Π_{srCOM} is correct and that the firewall is both strongly transparent and anti-signalling.

The former follows from the firewall only rerandomising the used randomness as is shown in Lemma 1.

Lemma 1. Π_{srCOM} is correct.

Proof. The protocol Π_{srCOM} is correct if the verifier $\mathcal{V}$ opens the received commitment c' under the received decommitment information d' to the committed-to value v of the sender S . Therefore, $\mathcal{V}$ checks that $c' = \text{COM}(v, d')$ with $d' = r + d$ set by RF . It holds that

$$\begin{aligned}
 c' &= \text{COM}(0, r) \cdot c \\
 &= \text{COM}(0, r) \cdot \text{COM}(v, d) \\
 &= (g^r \cdot T_1^0, h^r \cdot T_2^0) \cdot (g^d \cdot T_1^v, h^d \cdot T_2^v) \\
 &= (g^r \cdot (g^d \cdot T_1^v), h^r \cdot (h^d \cdot T_2^v)) \\
 &= (g^{r+d} \cdot T_1^v, h^{r+d} \cdot T_2^v) \\
 &= (g^{d'} \cdot T_1^v, h^{d'} \cdot T_2^v) \\
 &= \text{COM}(v, d').
 \end{aligned}$$

It remains to show the transparency and anti-signalling properties of the firewall. Since the firewall of the verifier only passes through the message, only the firewall of the sender has to be analysed. As was shown above, the firewall of the sender solely changes the outgoing commitment c to $c' = (g^{r+d} \cdot T_1^v, h^{r+d} \cdot T_2^v)$. As one can see, this changes the used randomness from d to $r + d$.

Lemma 2. RF in Π_{srCOM} is strongly transparent.

Proof. Recall that strong transparency means that for all environments, it is indistinguishable whether an incorruptible party or an incorruptible party composed with a firewall is used. For an incorruptible party, d is independent and uniformly random. Thus, the firewall of the sender adding an independent uniform random element r to it does not change its distribution. As this is the only change to the code by the firewall and the code of the party does not have to be changed to accommodate the firewall, the strong transparency property holds.

Lemma 3. RF in Π_{srCOM} is anti-signalling.

Proof. The definition of an anti-signalling firewall is that it is indistinguishable for all environments whether a composed party is used that consists of an incorruptible party and an incorruptible firewall or one consisting of a corruptible party and an incorruptible firewall. If the party were to be corrupted, this would only imply that d was output by the random function of the adversary. However, the sum $r + d$ would still be independent uniformly random because of the randomness added by the incorruptible firewall. As the randomness function is the only part that can be changed by the adversary, it follows that the firewall is anti-signalling.

The theorem follows.

7 Oblivious Transfer

Oblivious transfer is one of the open problems stated by [26] and [24]. Independently from this work, such a subversion-resilient OT was provided by Chakraborty et al. [25] in the aforementioned subversion model of [26]. While much more involved in the model of Chakraborty et al., we show that a relatively simple and efficient OT protocol already suffices for subversion-resilience in our plain UC model.

The sender S who knows two messages m_0, m_1 communicates with the receiver $\mathcal{R}$ who has a choice bit b and wants to learn m_b . The goal of the interaction is to allow the receiver to learn this message, and this message only, without the sender learning anything about b . The OT protocol consists of two rounds with each party sending one message to each other. As both of the messages are generated using randomness, this requires two different firewalls.

7.1 Ideal Functionality: Oblivious Transfer

The ideal functionality of the OT is the following:

$\mathcal{F}_{\text{OT}}$

Two-party ideal oblivious transfer protocol between a sender S and a receiver $\mathcal{R}$.

- Upon receiving an input (**OT-send**, $sid, \mathcal{R}, (m_0, m_1)$) from S , where $m_i \in \mathbb{G}$, record the tuple (sid, m_0, m_1) and generate a public delayed output (**OT-sent**, $sid, \mathcal{R}$) to S . Ignore subsequent commands of the form (**OT-send**, $sid, \mathcal{R}, \cdot$).
- Upon receiving an input (**OT-receive**, sid, S, b) from $\mathcal{R}$, where $b \in \{0, 1\}$, ignore the message if (sid, m_0, m_1) is not recorded. Otherwise, record the tuple (sid, b) and generate a public delayed output (**OT-received**, sid, S, m_b) to $\mathcal{R}$. Ignore subsequent commands of the form (**OT-receive**, $sid, S, \cdot$).
- \\ Corruption Handling:
 - On input (**secret**, $sid, \mathcal{P}, \text{sec}_{\mathcal{P}}$), store (**secret**, $sid, \mathcal{P}, \text{sec}_{\mathcal{P}}$). If $\mathcal{P}$ is semi-honestly or maliciously corrupted, send (**secret**, $sid, \mathcal{P}, \text{sec}_{\mathcal{P}}$) to Sim .
 - Ignore all further inputs.

We base our protocol on the UC-secure 1-out-of-2 OT protocol given by [21] where one of the messages $m_0, m_1 \in \mathbb{G}$ from the sender S can be obtained by the receiver $\mathcal{R}$. The protocol has to be adapted by making the field element $T_2 \in \mathbb{G}$ part of its CRS. The group $\mathbb{G}$ with generator g and the field $\mathbb{Z}_q$ are public inputs. While security under augmented semi-honest corruption (i.e., semi-honest

corruption where the adversary can choose the input of the corrupted party) is given, plain semi-honest security of the base OT scheme in [21] has not been shown yet. The real run of the oblivious transfer scheme is illustrated in Fig. 6.

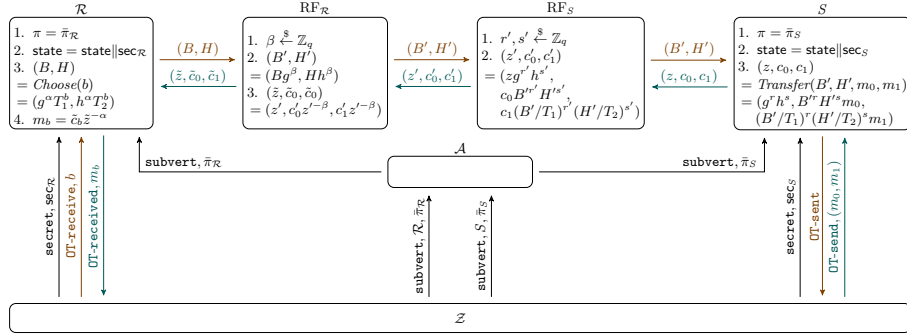


Fig. 6: Real run of the oblivious transfer scheme (omitting $\mathcal{F}_{\text{CRS}}$ and $\mathcal{F}_{\text{AUTH}}$). Here, $\text{Choose}(b)$ is used to embed the choice bit b , while $\text{Transfer}(B', H', m_0, m_1)$ then embeds the values m_0, m_1 to be transferred.

$\Pi_{\text{srOT}}^{\mathcal{F}_{\text{CRS}}, \mathcal{F}_{\text{AUTH}}}$

Realises:

Subversion-resilient two-party oblivious transfer protocol between a main party sender S , its sub-party firewall RF_S , a main party receiver $\mathcal{R}$, its sub-party firewall $RF_{\mathcal{R}}$, and an adversary $\mathcal{A}$.

Parameters:

- Functionality $\mathcal{F}_{\text{AUTH}}$.
- Functionality $\mathcal{F}_{\text{CRS}}$:
On sending $(\text{value}, \text{sid})$ to $\mathcal{F}_{\text{CRS}}$, every party receives $(\text{crs}, \text{sid}, (g, T_1, h, T_2))$.
- Every main-party has an empty initialised secret tape.

Behaviour of Party $\mathcal{R}$

\\ Secret Handling:

- Upon receiving $(\text{secret}, \text{sid}, \text{sec})$ from $\mathcal{Z}$, write sec on the secret tape.

\\ Oblivious Transfer Handling:

- Upon receiving input $(\text{OT-receive}, \text{sid}, S, b)$ from $\mathcal{Z}$, where $b \in \{0, 1\}$, get α from the randomness function, \\ presumably subverted and set $(B, H) := \text{Choose}(g, h, T_1, T_2, b) = (g^\alpha \cdot T_1^b, h^\alpha \cdot T_2^b)$. Send $(\text{send}, \text{sid}, \mathcal{R}, S, (B, H))$ to $RF_{\mathcal{R}}$ via an immediate channel.
- Upon receiving $(\text{sent}, \text{sid}, \mathcal{R}, S, (\tilde{z}, \tilde{c}_0, \tilde{c}_1))$ from $RF_{\mathcal{R}}$, set $m_b := \tilde{c}_b \cdot \tilde{z}^{-\alpha}$. Output $(\text{OT-received}, \text{sid}, S, m_b)$ to $\mathcal{Z}$.

Behaviour of Sub-Party $RF_{\mathcal{R}}$

- Upon receiving $(\text{send}, \text{sid}, \mathcal{R}, S, (B, H))$ from $\mathcal{R}$, sample $\beta \xleftarrow{\$} \mathbb{Z}_q$, set $(B', H') := (B \cdot g^\beta, H \cdot h^\beta)$, and send $(\text{send}, \text{sid}, \mathcal{R}, S, (B', H'))$ in the name of $\mathcal{R}$ to $\mathcal{F}_{\text{AUTH}}$.
- Upon receiving $(\text{send}, \text{sid}, S, \mathcal{R}, (z', c'_0, c'_1))$ from $\mathcal{F}_{\text{AUTH}}$, set $(\tilde{z}, \tilde{c}_0, \tilde{c}_1) := (z', c'_0 \cdot z'^{-\beta}, c'_1 \cdot z'^{-\beta})$, and send $(\text{send}, \text{sid}, S, \mathcal{R}, (\tilde{z}, \tilde{c}_0, \tilde{c}_1))$ via an immediate channel to $\mathcal{R}$.

Behaviour of Party S

$\backslash\backslash$ Secret Handling same as for Party $\mathcal{R}$.

$\backslash\backslash$ Oblivious Transfer Handling:

- Upon receiving $(\text{send}, \text{sid}, \mathcal{R}, S, (B', H'))$ from RF_S , output $(\text{OT-send}, \text{sid}, \mathcal{R})$ to $\mathcal{Z}$.
- Upon receiving $(\text{OT-send}, \text{sid}, \mathcal{R}, (m_0, m_1))$ from $\mathcal{Z}$, where $m_0, m_1 \in \mathbb{G}$, get r, s from randomness function, $\backslash\backslash$ presumably subverted and set $(z, c_0, c_1) := \text{Transfer}(g, h, T_1, T_2, B', H', m_0, m_1) = (g^r \cdot h^s, B'^r \cdot H'^s \cdot m_0, (B'/T_1)^r \cdot (H'/T_2)^s \cdot m_1)$. Send $(\text{send}, \text{sid}, S, \mathcal{R}, (z, c_0, c_1))$ to RF_S via an immediate channel.

Behaviour of Sub-Party RF_S

- Upon receiving $(\text{send}, \text{sid}, \mathcal{R}, S, (B', H'))$ from $\mathcal{F}_{\text{AUTH}}$, send $(\text{send}, \text{sid}, \mathcal{R}, S, (B', H'))$ via an immediate channel to S .
- Upon receiving $(\text{send}, \text{sid}, S, \mathcal{R}, (z, c_0, c_1))$ from S , sample $r', s' \xleftarrow{\$} \mathbb{Z}_q$, set $(z', c'_0, c'_1) := (z \cdot g^{r'} \cdot h^{s'}, c_0 \cdot B'^{r'} \cdot H'^{s'}, c_1 \cdot (B'/T_1)^{r'} \cdot (H'/T_2)^{s'})$, and send $(\text{send}, \text{sid}, S, \mathcal{R}, (z', c'_0, c'_1))$ in the name of S to $\mathcal{F}_{\text{AUTH}}$.

7.2 Security Proof

This protocol can be shown to fulfil the following:

Theorem 3. *The protocol Π_{srOT} UC-realises the ideal functionality $\mathcal{F}_{\text{OT}}$ in the $\{\mathcal{F}_{\text{CRS}}, \mathcal{F}_{\text{AUTH}}\}$ -hybrid model, where the main parties are maliciously, semi-honestly or subversion corrupted with a subversion code $\bar{\pi}$, and the respective sub-parties RF are honest, semi-honest or malicious.*

Proof. The proof follows the steps described in Section 5.3. From Corollary 1, we know that if Π_{srOT} UC-emulates Π_{OT} and Π_{OT} UC-realises $\mathcal{F}_{\text{OT}}$, then Π_{srOT} UC-realises $\mathcal{F}_{\text{OT}}$. In [21], it was proved that a variant of Π_{OT} without T_2 in the CRS UC-realises $\mathcal{F}_{\text{OT}}$. For our proof, we, hence, only have to focus on the former part of Π_{srOT} UC-emulating Π_{OT} . Here, we have to show that Π_{srOT} is correct and that the firewall is both strongly transparent and anti-signalling.

The former follows from the firewall only rerandomising the used randomness as is shown in Lemma 4.

Lemma 4. *Π_{srOT} is correct.*

Proof. The protocol Π_{srOT} is correct if the receiver $\mathcal{R}$ can open the chosen message m_b , that is $m_b = \tilde{c}_b \cdot \tilde{z}^{-\alpha}$. We consider the cases separately:

– Case $b = 0$:

$$\begin{aligned}
\tilde{c}_0 \cdot \tilde{z}^{-\alpha} &= c'_0 \cdot z'^{-\beta} \cdot z'^{-\alpha} \\
&= B'^{r+r'} \cdot H'^{s+s'} \cdot m_0 \cdot g^{-(\alpha+\beta)(r+r')} \cdot h^{-(\alpha+\beta)(s+s')} \\
&= g^{(\alpha+\beta)(r+r')} \cdot h^{(\alpha+\beta)(s+s')} \cdot m_0 \cdot g^{-(\alpha+\beta)(r+r')} \cdot h^{-(\alpha+\beta)(s+s')} \\
&= m_0.
\end{aligned}$$

– Case $b = 1$:

$$\begin{aligned}
\tilde{c}_1 \cdot \tilde{z}^{-\alpha} &= c'_1 \cdot z'^{-\beta} \cdot z'^{-\alpha} \\
&= (B'/T_1)^{r+r'} \cdot (H'/T_2)^{s+s'} \cdot m_1 \cdot g^{-(\alpha+\beta)(r+r')} \cdot h^{-(\alpha+\beta)(s+s')} \\
&= g^{(\alpha+\beta)(r+r')} \cdot h^{(\alpha+\beta)(s+s')} \cdot m_1 \cdot g^{-(\alpha+\beta)(r+r')} \cdot h^{-(\alpha+\beta)(s+s')} \\
&= m_1.
\end{aligned}$$

It remains to show the transparency and anti-signalling properties of the firewall. For that, we consider the firewalls for the sender and the receiver separately.

Sender The firewall of the sender only changes the outgoing tuple (z, c_0, c_1) to

$$\begin{aligned}
(z', c'_0, c'_1) &= (z \cdot g^{r'} \cdot h^{s'}, c_0 \cdot B'^{r'} \cdot H'^{s'}, c_1 \cdot (B'/T_1)^{r'} \cdot (H'/T_2)^{s'}) \\
&= (g^{r+r'} \cdot h^{s+s'}, B'^{r+r'} \cdot H'^{s+s'} \cdot m_0, (B'/T_1)^{r+r'} \cdot (H'/T_2)^{s+s'} \cdot m_1).
\end{aligned}$$

As one can see, this changes the used randomness from r and s to $r + r'$ and $s + s'$.

Lemma 5. RF_S in Π_{srOT} is strongly transparent and anti-signalling.

Proof. Strong transparency requires that for all environments, it is indistinguishable whether an incorruptible party or an incorruptible party composed with a firewall is used. Because we consider an incorruptible party, r and s are independent and uniformly random. Thus, RF_S adding independent uniform random elements r', s' to r, s does not change the distribution. As this is the only change to the code by the firewall and the code of the party does not have to be changed to accommodate the firewall, the strong transparency property holds.

The definition of an anti-signalling firewall is that it is indistinguishable for all environments whether a composed party is used that consists of an incorruptible party and an incorruptible firewall or one consisting of a corruptible party and an incorruptible firewall. Even if the sender were to be corrupted and r and s were output by the random function of the adversary, the sums $r + r'$ and $s + s'$ would still be independent uniform random because of the randomness added by the incorruptible firewall. As the randomness function is the only part that can be changed by the adversary, it follows that the firewall is anti-signalling.

Receiver The firewall $RF_{\mathcal{R}}$ changes both the tuple (B, H) output by the receiver to $(B', H') = (B \cdot g^\beta, H \cdot h^\beta) = (g^{\alpha+\beta} \cdot T_1^b, h^{\alpha+\beta} \cdot T_2^b)$ and (z', c'_0, c'_1) to

$$\begin{aligned} (\tilde{z}, \tilde{c}_0, \tilde{c}_1) &= (z', c'_0 \cdot z'^{-\beta}, c'_1 \cdot z'^{-\beta}) \\ &= (g^{r+r'} \cdot h^{s+s'}, B^{r+r'} \cdot H^{s+s'} \cdot m_0, (B/T_1)^{r+r'} \cdot (H/T_2)^{s+s'} \cdot m_1). \end{aligned}$$

Again, one can see that this only changes the used randomness from α to $\alpha + \beta$, which is reverted back before the message from the sender is passed to the receiver.

Lemma 6. *$RF_{\mathcal{R}}$ in Π_{srOT} is strongly transparent and anti-signalling.*

Proof. The fact that the message from the sender to the receiver is changed to equal the one in the underlying protocol implies that this message has the same distribution. Hence, only the first message has to further be analysed.

For the transparency property, we only consider incorruptible parties. Hence, we know α and β to be independent and uniformly random and, therefore, their sum is as well and does not change the output distribution. As this is the only change to the code by the firewall and the code of the party does not have to be changed to accommodate the firewall, the strong transparency property holds.

Even if the receiver were to be corrupted and α was output by the random function of the adversary, the sum $\alpha + \beta$ would still be independent uniform random because of the randomness added by the incorruptible firewall. As the randomness function is the only part that can be changed by the adversary, it follows that the firewall is anti-signalling.

The theorem follows.

8 Circumvention of an Impersonation Attack

In the previous sections, we showed that an honest firewall can indeed completely remove the leakage given by a subverted implementation. Hence, the security level of the system is improved in this case. However, while our analysis shows that all corruption cases are simulatable, a malicious firewall can impersonate the combined party even if the main party itself is honest. This is consistent with the handling by Chakraborty et al. [26]. The attentive reader might now ask why they should use such a firewall as the security in one scenario improves, while the security in another scenario declines drastically. In this section, we will first present a careful analysis that will show that the only damage which could be caused by such a corrupted firewall lies in the danger of *impersonation attacks* and not in the leaking of secret information. Then, we will discuss different approaches of how to handle this situation. In the first approach, we will add an ideal functionality $\mathcal{F}_{\text{chAUTH}}$, similar to the sanitised authenticated channels used by Chakraborty et al. This functionality exactly covers the adversarial capabilities provided by a corrupted firewall. We describe how to add such a functionality to our commitment protocol to protect against all impersonation

attacks. Afterwards, we show two realisations of these functionality, first a very efficient one for our commitment protocol based on signatures and then a more general version using zero-knowledge proofs for our oblivious transfer protocol. The general version is also applicable to a wide range of other protocols. We note that in [26], the employment of a three-tier model is discussed where a third component is added to the party and firewall. This could also be used in our model. Finally, we discuss that many security guarantees might still be guaranteed even in the presence of an attacker with impersonation capabilities. We note that all adaptations or extensions to the base protocols presented here mean that we lose the transitivity of the UC-emulation. Hence, we either need to directly show the simulation in the ideal world or need to take a two-step approach. To simplify our discussion, we again focus on a two-party scenario between two parties named Alice and Bob, where the honest party Alice is using a maliciously corrupted firewall.

8.1 On the Corruption Transition Table 2

For the analysis of Theorem 1, we first have to observe that as the firewall handles all communication to the other parties, a maliciously corrupted firewall can not only isolate its party from the others, but could also impersonate it by sending messages in its name. To protect against this, adjustments to the protocol are required, e.g., by adding an authenticated communication level, which might get certain security properties back. Hence, the corruption transition table is dependent on the specific protocol. This emphasises why we will show the simulatable indistinguishability in the real world instead of transferring the corruption transition behaviour into the ideal world.

Some transition cases could be seen as too strong as some security guarantees which are expected in Π are not given in Π' . For example, an honest or subverted party $\mathcal{P}$ would not lose the binding property of its commitment in Π but the composed corruption behaviour in Π' would give the content of the commitment to the adversary. We show what is achievable with the notions of UC-simulation. That not all desirable security guarantees are captured will be discussed further in Section 8.5. The notably achievable double state of “honest/malicious” for an honest $\mathcal{P}$ and a malicious RF is the result of an important insight: If we do not want to adapt given protocols in a complex way, we can only hope for the composed classical behaviour of a malicious party. But the main aspect why it behaves maliciously in the intuitive understanding is that RF can impersonate $\mathcal{P}$. This is naturally given in the composed behaviour, where $\mathcal{A}$ can impersonate $\mathcal{P}$. If we add one of the solutions discussed below to the protocol, we can achieve and prove via UC-simulation that the composed party behaves honestly.

8.2 Impersonation Attacks

In this section, we first want to understand the capabilities of a corrupted firewall. The importance of this was already mentioned explicitly by Dodis et al.

[33] *Second, and more importantly, this definitional choice provides an elegant solution to a natural concern about reverse firewalls: What happens when the firewall itself is corrupted? Of course, if both Alices own machine and her firewall are compromised, then we cannot possibly hope for security. But, if Alices own implementation is correct and the firewall has been corrupted, then we can view the firewall as “part of” the adversary in the firewall-free protocol between Alice and Bob. Since this underlying protocol must itself be secure, it trivially remains secure in the presence of a corrupted firewall.*

As indicated by the quote above, the problem could easily be handled using game-based approaches by treating the corrupted firewall as part of the other party. Our notion of transparency also allows us to treat the combination of the firewall and the *outside* parties as a maliciously corrupted party and the usual security guarantees against such parties thus imply security. Hence, even if using a maliciously corrupted firewall which could isolate the party from the other one, no security would be lost. Note that this argument heavily requires transparency of the firewall. In contrast, Chakraborty et al. who also introduce the corruption case of *isolation*, are not able to make use of the security guarantees of the underlying protocol in a black-box manner, as their solution has a feedback channel and the party thus needs to communicate explicitly with the firewall. Hence, as shown in the following quote, they are not able to provide any security guarantee in this scenario.

[26] *The Isolated case corresponds to the situation where the core is honest and the firewall is corrupted, and thus the firewall is isolating the core from the network. This will typically correspond to a corrupted party. However, in some cases, some partial security might be obtainable, like the inputs of the core being kept secret.*

As described above, the notion of strong transparency intuitively allows to guarantee that Alice’s secret is not leaked even if the firewall is maliciously corrupted by treating the firewall as part of a maliciously corrupted Bob. While this argument is true in the context of the protocols studied by Mironov and Stephens-Davidowitz [46] and Dodis, Mironov and Stephens-Davidowitz [33], it does *not* hold in general. Careful consideration shows that the protocols studied in the previous works did not require any form of *authentication*. However, as the complete communication between the parties needs to be modelled in the UC framework, questions about authentication need to be dealt with (typically by using the ideal functionality $\mathcal{F}_{\text{AUTH}}$). Hence, when studying the capabilities of a corrupted firewall in the UC model, we need to consider the following four points:

1. One can easily see that we cannot guarantee that the system is non-signalling as the firewall can just pass on the output of the party (in case of a subverted party).
2. Additionally, the firewall can either isolate the party from the protocol or send messages in its name.

3. Hence, when running a priorly given protocol, the firewall could impersonate its party.
4. But can we still provide some guarantees of the security of the underlying protocol?

To make this discussion more explicit, let us take a look at the example of a commitment scheme:

In the case of the commitment scheme (presented in Section 6), from the viewpoint of the other parties, a maliciously corrupted RF ...

1. ... could pass through the output from Alice, thus revealing the leakage if subverted.
2. ... could rerandomise the output from Alice.
3. ... could isolate Alice from the protocol by not sending anything, though this would probably soon be detected by Bob or a higher protocol.
4. ... could impersonate Alice by sending a new commitment containing some chosen message.

If Alice is honest, then her output would not leak any secret information and the first case would guarantee a completely honest combined party. Similarly, the combined party would also behave honestly in the second case, as the malicious firewall could only rerandomise the uniformly generated randomness, which would still lead to uniformly generated randomness. We can ignore the third case, as a lack of communication will usually not invalidate any security guarantees. In the fourth case, however, the corrupted firewall (and thus the attacker) can send messages in the name of Alice, which is clearly a non-honest behaviour and the combined party thus needs to be treated as corrupted.

We note that a dedicated weaker corruption setting of an impersonation could suffice instead of the overstatement of a maliciously corrupted combined party. Hence, in order to prevent the security problems due to a corrupted firewall, it is sufficient to solve the impersonation problem.

8.3 Adding $\mathcal{F}_{\text{chAUTH}}$ to Π_{srCOM}

We show how to handle the impersonation problem by considering a new ideal functionality $\mathcal{F}_{\text{chAUTH}}$. The main idea of this functionality is to handle the fourth case described above, where the maliciously corrupted firewall itself chooses a message and sends it in the name of the party. The goal of $\mathcal{F}_{\text{chAUTH}}$ is to guarantee via a *check* that the output of the firewall is a rerandomised version of the output of Alice, i.e., to force the malicious firewall to behave semi-honestly. More generally, we assume that if Alice outputs x , then an honest firewall would output a value y such that $(x, y) \in R$ for some NP-relation R . We denote a witness proving that $(x, y) \in R$ by w and assume that the honest firewall produces such a witness when generating y . For the sake of simplicity, we directly combine this

functionality with our authenticated channels. However, in the following, we will only explain the changes caused by this additional check. After the functionality sends the backdoor message (**send**, $sid, \mathcal{P}_1, \mathcal{P}_2, y$) to $\mathcal{A}$, it continues as in $\mathcal{F}_{\text{AUTH}}$ given in Definition 3. We note that one might also be able to more formally frame the isolation case of [26] using our idea of the ideal functionality $\mathcal{F}_{\text{chAUTH}}$.

$\mathcal{F}_{\text{chAUTH}_R}$

Provides:

Authenticated channel between $\mathcal{P}_1$ and $\mathcal{P}_2$ that checks whether the output y of the firewall of $\mathcal{P}_1$ and the output x of $\mathcal{P}_1$ fulfil $(x, y) \in R$.

Behaviour:

– Interface IO

- Upon invocation with input (**check**, sid, x) from $\mathcal{P}_1$, record the tuple (sid, x) and return (**checked**, sid, OK) to $\mathcal{P}_1$. Ignore subsequent commands of the form (**check**, sid, x).
- On input (**send**, $sid, (y, w)$) from RF , ignore the message if (**check**, sid, x) is not recorded. If $(x, y) \notin R$ (using w), ignore the message. If $(x, y) \in R$ (using w), send backdoor message (**send**, $sid, \mathcal{P}_1, \mathcal{P}_2, y$) to $\mathcal{A}$. Ignore subsequent commands of the form (**send**, $sid, (y, w)$).

In the concrete case of our commitment protocol, x would correspond to a commitment c and the relation R corresponds to all y that are rerandomisations of c , i.e.,

$$R = \{(c, c') \mid \exists r' \in \{0, 1\}^* : c' = \text{COM}(0, r) \cdot c\}.$$

Finally, the witness w is simply the random string r' used by the firewall to rerandomise c .

In the following, we show how to construct a simulator handling the case that the party Alice is honest and the firewall is maliciously corrupted in the presence of $\mathcal{F}_{\text{chAUTH}}$. This simulator shows that we can treat this case as though as Alice is honest and the firewall is not present.

Execution in the Real World: To commit a value, the execution in the real world is as follows: First, the environment $\mathcal{Z}$ sends some value v to Alice who then computes a corresponding commitment c using randomness r . This commitment c is then given to $\mathcal{F}_{\text{chAUTH}}$ that returns **OK** to Alice. Now, Alice sends c to the maliciously corrupted firewall who forwards it to the environment $\mathcal{Z}$. The environment now sends some commitment c' and the randomness r' to $\mathcal{F}_{\text{chAUTH}}$ (in the name of the dummy attacker). The ideal functionality $\mathcal{F}_{\text{chAUTH}}$ now checks whether $c' = \text{COM}(0, r') \cdot c$ and returns c' to the attacker (resp. the environment). Now, if $\mathcal{Z}$ sends **OK**, $\mathcal{F}_{\text{chAUTH}}$ sends c' to the second party Bob.

To open a value, the execution in the real world is as follows: First, the environment $\mathcal{Z}$ sends **open** to Alice who then sends the pair (v, r) to $\mathcal{F}_{\text{chAUTH}}$ and is given **OK**. Now, Alice sends (v, r) to the maliciously corrupted firewall, who

forwards this pair to the environment (via the dummy attacker). The environment $\mathcal{Z}$ now sends a pair $(\bar{v}, \bar{r})$ to $\mathcal{F}_{\text{chAUTH}}$ who returns it directly to the environment. After $\mathcal{Z}$ sends OK, the functionality $\mathcal{F}_{\text{chAUTH}}$ sends $(\bar{v}, \bar{r})$ to Bob.

Simulation in the Ideal World: To handle the commitment, our simulator Sim works as follows: First, $\mathcal{Z}$ sends v to Alice, who then calculates the corresponding commitment c using randomness r . This commitment c is then given to $\mathcal{F}_{\text{chAUTH}}$ that returns OK to Alice. Now, Alice send c to the adversary, which is the simulator Sim . The simulator now sends c in the name of the malicious firewall to the dummy attack which outputs it to $\mathcal{Z}$. The environment now sends a commitment c' and the randomness r' in the name of the firewall to Sim . The simulator now checks whether $c' = \text{COM}(0, r') \cdot c$ and sends c' to the dummy attacker, who forwards it to $\mathcal{Z}$. If $c' \neq \text{COM}(0, r') \cdot c$, the simulator ignores the message. If the environment $\mathcal{Z}$ sends OK, the simulator forwards c' to Bob.

To handle the opening, our simulator Sim works as follows: First, $\mathcal{Z}$ sends **open** to Alice who then sends (v, r) to $\mathcal{F}_{\text{chAUTH}}$. The pair (v, r) is then forwarded to the attacker, which is Sim . The simulator sends (v, r) to the environment (using the dummy attacker controlling the corrupted firewall). The environment then sends $(\bar{v}, \bar{r})$ to the simulator, which forwards it to the dummy attacker and then to the environment. If the environment $\mathcal{Z}$ sends OK, the simulator forwards $(\bar{v}, \bar{r})$ to Bob.

8.4 Circumventing Impersonation on Protocol Side

As shown above, the ideal functionality $\mathcal{F}_{\text{chAUTH}}$ can be used to prevent impersonation attacks. In this section, we will show two possible ways how to concretely realise these ideal functionality exemplarily for both commitments (Section 6) and oblivious transfer (Section 7). Our solution for commitments will be very efficient, but specifically tailored to commitments. In contrast, our solution for oblivious transfer is not as efficient, but very general and should work for all protocols.

Protecting Commitments Through Signatures While we constructed bit commitments in Section 6, we will now consider string commitments. We note that the composition theorem simply allows us to build string commitments out of bit commitments by combining the single bit commitments. In the following, we will now make use of *unique signatures*, i.e., a signature scheme where each message has exactly one valid signature. See, e.g., [45, 38] for more information and constructions of such signatures. We assume that Alice holds some signing key sk and Bob holds the verification key vk which was distributed, e.g., via a public-key infrastructure. If Alice wants to commit to a value v , we first compute such unique signature, denoted as $\sigma(v)$. Since the signatures are unique, they contain no randomness and, therefore, cannot leak information about the secret. Then, we compute the string commitment c of the value $\bar{v} = v \parallel \sigma(v)$. After receiving the commitment c' and once it is opened, Bob will perform an additional verification step using Vfy to check whether the signature $\sigma(v)$ contained in the commitment

$c' = \text{COM}(v \parallel \sigma(v))$ is a valid signature of the value v . Now, if the adversary sends a commitment which passes the verification check of Bob, we know that it is a rerandomisation of c . Otherwise, c' would contain a forgery which would contradict the security of the signature scheme.

Protecting Oblivious Transfer Through Zero-Knowledge Proofs Oblivious transfer is a slightly more difficult protocol than commitments, as neither the information of which message was chosen, b , nor the message not chosen, m_{1-b} , will be revealed which makes our above solution not directly transferable. Nevertheless, we will also make use of signature schemes, which in this setting do not need to be unique. We again assume that Alice holds some signing key sk and Bob holds the verification key vk which was distributed, e.g., via a public-key infrastructure. Whenever a party sends the value x , it will also send a signature σ of x along with it. When the firewall is given x and σ , it first produces the output y along with the witness w for $(x, y) \in R$ as well as a zero-knowledge proof ρ of the language

$$\mathcal{L} = \{(x, \sigma, w) \mid (x, y) \in R \text{ via } w \wedge \forall y (vk, x, \sigma) = 1\}.$$

The firewall now sends y along with ρ . When obtaining y and ρ , the receiving party can verify that y is a valid rerandomisation by checking ρ . Due to the zero-knowledge property of ρ , the receiving party does not learn anything about the (possibly leaking) values x and σ . The soundness property of ρ guarantees that ρ contains a valid proof and thus, in turn, a valid value-signature pair (x, σ) . To produce a valid proof for y , the firewall can thus only rerandomise x .

8.5 Remaining Security Guarantees in the Presence of an Impersonation

By using solutions to avoid the impersonation problems we can avoid having to treat the combined party as maliciously corrupted. However, it comes with a cost as we need to modify the underlying protocol when realising this functionality. Without any changes to the protocol, it seems unlikely that we can handle the impersonation problem. However, some security guarantees might still be maintained, hence providing a graceful degradation. This can, for example, be of interest to Alice if she primarily wants to secure her committed message.

Let us again consider the situation of a commitment: Since the rerandomisation check was not added to $\mathcal{F}_{\text{AUTH}}$ itself, a simulation is no longer possible. The simulator does not know whether the output c' of a firewall is a rerandomisation of the original commitment c or a completely new commitment due to the hiding property of the commitment scheme. Nevertheless, a careful analysis shows that both the hiding and binding of Alice's commitment still hold in this case, as the malicious firewall is neither able to look into the original commitment c nor to open it to a different value. The only possible attacks are that the value given to Bob is either wrong or that Bob does not receive any message at all. This also extends to other functionalities like the coin toss which is still efficiently realisable.

It is interesting to see that we cannot capture all security guarantees by UC-simulation. The impersonation corruption might be interesting for further research as, for example, in the case of Alice being honest and her firewall being malicious, this would not result in losing the hiding property, even if we treat the composed party as malicious. We could also circumvent the impersonation by giving the simulator more power such that it can still impersonate in the opening phase. However, this would imply that the ideal commitment is equivocable.

Hence, if the goal is that the composed party behaviour should exactly represent the natural security guarantees of the non-composed setting, the UC model has to be adapted in extreme ways and even then, it would be rather unlikely that all security guarantees are simulated. This hints at limits of the (UC)-simulation as a gold standard for security analyses.

9 Conclusion

In this work, we provide a new model to analyse the security of given protocols against subversion attacks within the plain Universal Composability (UC) framework and show how to integrate reverse firewalls as a countermeasure. More specifically, our model effectively captures realistic subversion scenarios where only the randomness of a party can be subverted. Then, given a real world *base* protocol securely UC-realising an ideal functionality, we prove that our resulting subversion-resilient protocol UC-realises this ideal functionality as well. Thereby, our model allows to argue subversion-resilience of already existing UC-secure protocols without necessitating additional security proofs. Our results rely on new definitions of properties for reverse firewalls, mainly strong transparency and anti-signalling, enabling robust security guarantees even when these firewalls are compromised. To demonstrate the practicality of our model, we applied it to commitment schemes and oblivious transfer protocols.

Based on the contributions of this paper, several directions for future research remain open: A natural extension would be to explore the applicability of the model to other cryptographic protocols. In particular, ensuring subversion-resilience in post-quantum cryptographic settings would strengthen the future security landscape. A more detailed analysis of impersonation attacks—or other possibly opened attack vectors—by the reverse firewalls as well as extending the framework to handle adaptive adversaries who can dynamically corrupt and subvert different components of a system could also enhance its robustness.

Acknowledgments. We thank the anonymous reviewers from Asiacrypt 2024, PKC 2025, and ACNS 2025 for their valuable feedback which yield to this improved work. We thank Johannes Ottenhues for proofreading and for his helpful discussions and suggestions.

This work was supported by funding by the German Federal Ministry of Education and Research (BMBF) under the project “Sec4IoMT” (ID 16KIS1692 and 16KIS1693), and by KASTEL Security Research Labs.

References

1. Abdolmaleki, B., Fleischhacker, N., Goyal, V., Jain, A., Malavolta, G.: Steganography-Free Zero-Knowledge. In: TCC (1). Lecture Notes in Computer Science, pp. 143–172. Springer (2022) (cit. on p. 12). https://doi.org/10.1007/978-3-031-22318-1_6
2. Alwen, J., shelat, a., Visconti, I.: Collusion-Free Protocols in the Mediated Model. In: CRYPTO. Lecture Notes in Computer Science, pp. 497–514. Springer (2008) (cit. on p. 11). https://doi.org/10.1007/978-3-540-85174-5_28
3. Armour, M., Poettering, B.: Algorithm substitution attacks against receivers. Int. J. Inf. Sec. **21**(5), 1027–1050 (2022) (cit. on pp. 7, 18). <https://doi.org/10.1007/s10207-022-00596-5>
4. Arnold, P., Berndt, S., Müller-Quade, J., Ottenhues, A.: Protection Against Subversion Corruptions via Reverse Firewalls in the Plain Universal Composability Framework. In: ACNS 2025. Lecture Notes in Computer Science, pp. 510–539. Springer (2025) (cit. on p. 1). https://doi.org/10.1007/978-3-031-95764-2_20
https://doi.org/10.1007/978-3-031-95764-2_20
5. Ateniese, G., Magri, B., Venturi, D.: Subversion-Resilient Signature Schemes. In: CCS, pp. 364–375. ACM (2015) (cit. on pp. 7, 18). <https://doi.org/10.1145/2810103.2813635>
6. Badertscher, C., Ciampi, M., Kiayias, A.: Agile Cryptography: A Universally Composable Approach. In: TCC. Lecture Notes in Computer Science, pp. 480–509. Springer (2023) (cit. on p. 12). https://doi.org/10.1007/978-3-031-48624-1_18
7. Bellare, M., Jaeger, J., Kane, D.: Mass-surveillance without the State: Strongly Undetectable Algorithm-Substitution Attacks. In: CCS, pp. 1431–1440. ACM (2015) (cit. on pp. 3, 7, 18). <https://doi.org/10.1145/2810103.2813681>
8. Bellare, M., Paterson, K.G., Rogaway, P.: Security of Symmetric Encryption against Mass Surveillance. In: CRYPTO (1). Lecture Notes in Computer Science, pp. 1–19. Springer (2014) (cit. on pp. 3, 7, 8, 12, 18, 55). https://doi.org/10.1007/978-3-662-44371-2_1
9. Bemmman, P., Berndt, S., Chen, R.: Subversion-Resilient Signatures Without Random Oracles. In: ACNS (1). Lecture Notes in Computer Science, pp. 351–375. Springer (2024) (cit. on p. 12). https://doi.org/10.1007/978-3-031-54770-6_14
10. Bemmman, P., Berndt, S., Diemert, D., Eisenbarth, T., Jager, T.: Subversion-Resilient Authenticated Encryption Without Random Oracles. In: ACNS. Lecture Notes in Computer Science, pp. 460–483. Springer (2023) (cit. on p. 12). https://doi.org/10.1007/978-3-031-33491-7_17
11. Bemmman, P., Chen, R., Jager, T.: Subversion-Resilient Public Key Encryption with Practical Watchdogs. In: Public Key Cryptography (1). Lecture Notes in Computer Science, pp. 627–658. Springer (2021) (cit. on p. 12). https://doi.org/10.1007/978-3-030-75245-3_23
12. Berndt, S., Liskiewicz, M.: Algorithm Substitution Attacks from a Steganographic Perspective. In: CCS, pp. 1649–1660. ACM (2017) (cit. on pp. 3, 7, 18, 25). <https://doi.org/10.1145/3133956.3133981>
13. Berndt, S., Wichelmann, J., Pott, C., Traving, T., Eisenbarth, T.: ASAP: Algorithm Substitution Attacks on Cryptographic Protocols. In: AsiaCCS, pp. 712–726. ACM (2022) (cit. on pp. 7, 18). <https://doi.org/10.1145/3488932.3517387>
14. Bossuat, A., Bultel, X., Fouque, P., Onete, C., van der Merwe, T.: Designing Reverse Firewalls for the Real World. In: ESORICS (1). Lecture Notes in Computer Science,

- pp. 193–213. Springer (2020) (cit. on p. 11). https://doi.org/10.1007/978-3-030-58951-6_10
15. Broadnax, B., Koch, A., Mechler, J., Müller, T., Müller-Quade, J., Nagel, M.: Fortified Universal Composability: Taking Advantage of Simple Secure Hardware Modules. *IACR Cryptol. ePrint Arch.* (2018) (cit. on p. 21)
 16. Canetti, R.: Universally Composable Security. *J. ACM* **67**(5), 28:1–28:94 (2020) (cit. on pp. 4, 13, 14). <https://doi.org/10.1145/3402457>
 17. Canetti, R.: Universally Composable Security: A New Paradigm for Cryptographic Protocols. *IACR Cryptol. ePrint Arch.* (2000) (cit. on pp. 9, 15, 16, 21, 23, 29, 32–34). <https://doi.org/10.1109/SFCS.2001.959888>
 18. Canetti, R.: Universally Composable Security: A New Paradigm for Cryptographic Protocols. In: *FOCS*, pp. 136–145. IEEE Computer Society (2001) (cit. on pp. 4, 13, 14, 16). <https://doi.org/10.1109/SFCS.2001.959888>
 19. Canetti, R., Fischlin, M.: Universally Composable Commitments. In: *CRYPTO. Lecture Notes in Computer Science*, pp. 19–40. Springer (2001) (cit. on p. 15). https://doi.org/10.1007/3-540-44647-8_2
 20. Canetti, R., Hogan, K., Malhotra, A., Varia, M.: A Universally Composable Treatment of Network Time. In: *CSF*, pp. 360–375. IEEE Computer Society (2017) (cit. on p. 24). <https://doi.org/10.1109/CSF.2017.38>
 21. Canetti, R., Sarkar, P., Wang, X.: Efficient and Round-Optimal Oblivious Transfer and Commitment with Adaptive Security. In: *ASIACRYPT* (3). *Lecture Notes in Computer Science*, pp. 277–308. Springer (2020) (cit. on pp. 9, 35–37, 39–41). https://doi.org/10.1007/978-3-030-64840-4_10
 22. Chakraborty, S., Dziembowski, S., Nielsen, J.B.: Reverse Firewalls for Actively Secure MPCs. In: *CRYPTO* (2). *Lecture Notes in Computer Science*, pp. 732–762. Springer (2020) (cit. on p. 11). https://doi.org/10.1007/978-3-030-56880-1_26
 23. Chakraborty, S., Ganesh, C., Pancholi, M., Sarkar, P.: Reverse Firewalls for Adaptively Secure MPC Without Setup. In: *ASIACRYPT* (2). *Lecture Notes in Computer Science*, pp. 335–364. Springer (2021) (cit. on p. 11). https://doi.org/10.1007/978-3-030-92075-3_12
 24. Chakraborty, S., Ganesh, C., Sarkar, P.: Reverse Firewalls for Oblivious Transfer Extension and Applications to Zero-Knowledge. In: *EUROCRYPT* (1). *Lecture Notes in Computer Science*, pp. 239–270. Springer (2023) (cit. on pp. 10, 11, 39). https://doi.org/10.1007/978-3-031-30545-0_9
 25. Chakraborty, S., Magliocco, L., Magri, B., Venturi, D.: Key Exchange in the Post-snowden Era: Universally Composable Subversion-Resilient PAKE. In: *ASIACRYPT* (5). *Lecture Notes in Computer Science*, pp. 101–133. Springer (2024) (cit. on pp. 5, 10, 11, 39, 54). https://doi.org/10.1007/978-981-96-0935-2_4
 26. Chakraborty, S., Magri, B., Nielsen, J.B., Venturi, D.: Universally Composable Subversion-Resilient Cryptography. In: *EUROCRYPT* (1). *Lecture Notes in Computer Science*, pp. 272–302. Springer (2022) (cit. on pp. 3, 5, 7–11, 16, 18, 19, 21, 27, 35, 39, 43–45, 47, 54, 55, 57, 58). https://doi.org/10.1007/978-3-031-06944-4_10
 27. Checkoway, S., Niederhagen, R., Everspaugh, A., Green, M., Lange, T., Ristenpart, T., Bernstein, D.J., Maskiewicz, J., Shacham, H., Fredrikson, M.: On the Practical Exploitability of Dual EC in TLS Implementations. In: *USENIX Security Symposium*, pp. 319–335. USENIX Association (2014) (cit. on p. 3)
 28. Chen, R., Huang, X., Yung, M.: Subvert KEM to Break DEM: Practical Algorithm-Substitution Attacks on Public-Key Encryption. In: *ASIACRYPT* (2). *Lecture Notes in Computer Science*, pp. 98–128. Springer (2020) (cit. on pp. 7, 18). https://doi.org/10.1007/978-3-030-64834-3_4

29. Cheng, J., Wang, Y., Chen, R., Huang, X.: Subverting Cryptographic Protocols from a Fine-Grained Perspective- A Case Study on 2-Party ECDSA. In: ACISP (2). Lecture Notes in Computer Science, pp. 370–390. Springer (2024) (cit. on p. 7). https://doi.org/10.1007/978-981-97-5028-3_19
30. Chow, S.S.M., Russell, A., Tang, Q., Yung, M., Zhao, Y., Zhou, H.: Let a Non-barking Watchdog Bite: Cliptographic Signatures with an Offline Watchdog. In: Public Key Cryptography (1). Lecture Notes in Computer Science, pp. 221–251. Springer (2019) (cit. on p. 12). https://doi.org/10.1007/978-3-030-17253-4_8
31. Degabriele, J.P., Farshim, P., Poettering, B.: A More Cautious Approach to Security Against Mass Surveillance. In: FSE. Lecture Notes in Computer Science, pp. 579–598. Springer (2015) (cit. on pp. 7, 18). https://doi.org/10.1007/978-3-662-48116-5_28
32. Discussion about Kyber’s tweaked FO transform, (2023). <https://groups.google.com/a/list.nist.gov/g/pqc-forum/c/WFRDl8DqYQ4>. Discussion Thread on the PQC mailing list (cit. on p. 3).
33. Dodis, Y., Mironov, I., Stephens-Davidowitz, N.: Message Transmission with Reverse Firewalls - Secure Communication on Corrupted Machines. In: CRYPTO (1). Lecture Notes in Computer Science, pp. 341–372. Springer (2016) (cit. on pp. 3–5, 7, 8, 11, 18, 25, 45, 56). https://doi.org/10.1007/978-3-662-53018-4_13
34. Fischlin, M., Mazaheri, S.: Self-Guarding Cryptographic Protocols against Algorithm Substitution Attacks. In: CSF, pp. 76–90. IEEE Computer Society (2018) (cit. on p. 12). <https://doi.org/10.1109/CSF.2018.00013>
35. Ganesh, C., Magri, B., Venturi, D.: Cryptographic reverse firewalls for interactive proof systems. Theor. Comput. Sci. **855**, 104–132 (2021) (cit. on p. 11). <https://doi.org/10.1016/j.tcs.2020.11.043>
36. Goldreich, O., Micali, S., Wigderson, A.: How to Play any Mental Game or A Completeness Theorem for Protocols with Honest Majority. In: STOC, pp. 218–229. ACM (1987) (cit. on pp. 4, 7). <https://doi.org/10.1145/3335741.3335755>
37. Hazay, C., Lindell, Y.: Efficient Secure Two-Party Protocols - Techniques and Constructions. Springer (2010) (cit. on p. 33)
38. Hofheinz, D., Jager, T.: Verifiable Random Functions from Standard Assumptions. In: TCC (A1). Lecture Notes in Computer Science, pp. 336–362. Springer (2016) (cit. on p. 48). https://doi.org/10.1007/978-3-662-49096-9_14
39. Hofheinz, D., Shoup, V.: GNUC: A New Universal Composability Framework. J. Cryptol. **28**(3), 423–508 (2015) (cit. on p. 21). <https://doi.org/10.1007/s00145-013-9160-y>
40. Ishai, Y., Kushilevitz, E., Ostrovsky, R., Sahai, A.: Zero-Knowledge Proofs from Secure Multiparty Computation. SIAM J. Comput. **39**(3), 1121–1152 (2009) (cit. on p. 7). <https://doi.org/10.1137/080725398>
41. Lepinski, M., Micali, S., shelat, a.: Collusion-free protocols. In: STOC, pp. 543–552. ACM (2005) (cit. on p. 11). <https://doi.org/10.1145/1807406.1807497>
42. Li, G., Liu, J., Zhang, Z., Zhang, Y.: UC-Secure Cryptographic Reverse Firewall-Guarding Corrupted Systems with the Minimum Trusted Module. In: Inscript. Lecture Notes in Computer Science, pp. 85–110. Springer (2021) (cit. on p. 11). https://doi.org/10.1007/978-3-030-88323-2_5
43. Lin, Y., Chen, R., Wang, Y., Wang, B., Liu, L.: Substitution Attacks Against Sigma Protocols. In: CSS. Lecture Notes in Computer Science, pp. 192–208. Springer (2022) (cit. on pp. 7, 18). https://doi.org/10.1007/978-3-031-18067-5_14
44. Marchiori, D., Giron, A.A., do Nascimento, J.P.A., Custódio, R.: Timing analysis of algorithm substitution attacks in a post-quantum TLS protocol. In: Anais do XXI

- Simpósio Brasileiro em Segurança da Informação e de Sistemas Computacionais, pp. 127–140 (2021) (cit. on pp. 7, 18). <https://doi.org/10.5753/sbseg.2021.17311>
45. Micali, S., Rabin, M.O., Vadhan, S.P.: Verifiable Random Functions. In: FOCS, pp. 120–130. IEEE Computer Society (1999) (cit. on p. 48). <https://doi.org/10.1109/SFFCS.1999.814584>
 46. Mironov, I., Stephens-Davidowitz, N.: Cryptographic Reverse Firewalls. In: EUROCRYPT (2). Lecture Notes in Computer Science, pp. 657–686. Springer (2015) (cit. on pp. 3, 4, 8–11, 19, 25–27, 45, 56). https://doi.org/10.1007/978-3-662-46803-6_22
 47. Perlroth, N., Larson, J., Shane, S: Secret documents reveal NSA campaign against encryption. New York Times. September 5 (2013) (cit. on p. 3)
 48. Russell, A., Tang, Q., Yung, M., Zhou, H.: Cliptography: Clipping the Power of Kleptographic Attacks. In: ASIACRYPT (2). Lecture Notes in Computer Science, pp. 34–64 (2016) (cit. on pp. 8, 12). https://doi.org/10.1007/978-3-662-53890-6_2
 49. Russell, A., Tang, Q., Yung, M., Zhou, H.: Correcting Subverted Random Oracles. In: CRYPTO (2). Lecture Notes in Computer Science, pp. 241–271. Springer (2018) (cit. on p. 12). https://doi.org/10.1007/978-3-319-96881-0_9
 50. Russell, A., Tang, Q., Yung, M., Zhou, H.: Generic Semantic Security against a Kleptographic Adversary. In: CCS, pp. 907–922. ACM (2017) (cit. on p. 12). <https://doi.org/10.1145/3133956.3133993>
 51. Russell, A., Tang, Q., Zhu, J.: Crooked Indifferentiability of the Feistel Construction. In: ASIACRYPT (6). Lecture Notes in Computer Science, pp. 427–460. Springer (2024) (cit. on p. 18). https://doi.org/10.1007/978-981-96-0938-3_14
 52. Simmons, G.J.: The history of subliminal channels. IEEE J. Sel. Areas Commun. **16**(4), 452–462 (1998) (cit. on p. 3). <https://doi.org/10.1109/49.668969>
 53. Teseleanu, G.: Unifying Kleptographic Attacks. In: NordSec. Lecture Notes in Computer Science, pp. 73–87. Springer (2018) (cit. on pp. 7, 18). https://doi.org/10.1007/978-3-030-03638-6_5
 54. Young, A.L., Yung, M.: Kleptography: Using Cryptography Against Cryptography. In: EUROCRYPT. Lecture Notes in Computer Science, pp. 62–74. Springer (1997) (cit. on pp. 3, 7, 18). https://doi.org/10.1007/3-540-69053-0_6
 55. Young, A.L., Yung, M.: The Dark Side of "Black-Box" Cryptography, or: Should We Trust Capstone? In: CRYPTO. Lecture Notes in Computer Science, pp. 89–103. Springer (1996) (cit. on p. 3). https://doi.org/10.1007/3-540-68697-5_8
 56. Young, A.L., Yung, M.: The Prevalence of Kleptographic Attacks on Discrete-Log Based Cryptosystems. In: CRYPTO. Lecture Notes in Computer Science, pp. 264–276. Springer (1997) (cit. on pp. 7, 18). <https://doi.org/10.1007/BFb0052241>

A Comparison to Chakraborty et al.

In this section, we will give a more detailed comparison of our work and model to the model introduced by Chakraborty, Magri, Nielsen, and Venturi [26] and also used by Chakraborty, Magliocco, Magri, and Venturi [25].

The main difference in our view of subversion attacks can be summed up in the question of “which attacks are undetectable?”.

Let us reconsider the two subversion attacks against commitments described in Section 1: In the first attack, *rejection sampling* was used to find good randomness such that $c = \text{Com}(v, r)$ reveals some information about the secret to the subverter.

In the second attack, the subverted code changed the input of the commitment and compute $c = \text{Com}(\text{sec}, r)$ instead.

In contrast to the model of Chakraborty et al. [26], we consider the second attack, i.e., subversions that change an input of the underlying algorithm, to be detectable. The reason for this is that the party running the subverted implementation might notice that the value in the commitment is not equal to the value that was supposed to be there by inspecting the output of the commitment. This kind of inspection closely resembles the *watchdogs*, introduced by Bellare, Paterson, and Rogaway [8]. Hence, the party might become suspicious and thus detect the presence of the subverted code.

This difference thus result in the major difference between our models. In our model, subversion attacks do not change the functionality of the underlying protocol and are simply implementation attacks. This allows our ideal world to have neither subverted implementations nor firewalls and lets us realise standard known functionalities that guarantee security without asking for firewalls. That the ideal world does not model any firewalls is possible as the environment cannot communicate with *RF* in the real world either. Hence, in our model, where only the randomness can be subverted, the analysed commitment functionality $\mathcal{F}_{\text{com}}$ used is simply the standard functionality, where the committing party sends a value v to commit to the functionality. Later on, it can also open this value to another party.

In contrast, in the model of Chakraborty et al. [26], the firewall does also exist in the ideal world and ideal functionalities thus need to have an explicit interface for the firewalls, called the *sanitisation interface*. For example, the commitment functionality $\mathcal{F}_{\text{com}}$ of Chakraborty et al. [26] thus works in three steps. First, the committing party sends value v to commit to the functionality. Then, the reverse firewall is informed that some values was sent and can send a blinding string r via the sanitisation interface of the functionality. The functionality now changes the stored value from v to $v \oplus r$ and informs the committing party about this by sending r . Finally, when the commitment is opened, the updated value $v \oplus r$ is revealed to the receiving party.

A.1 Transparency

In Section 4.2, we introduced the notions of *outer transparency* and *strong transparency*. Suppose that the parties $\mathcal{P}$ and $\mathcal{P}'$ communicate via some protocol Π where $\mathcal{P}$ might use a firewall. Informally, outer transparency means that $\mathcal{P}'$ cannot decide whether the party $\mathcal{P}$ uses the firewall or not. Strong transparency also requires something one might call *inner transparency*, where $\mathcal{P}$ itself does not need to be aware of the existence of the firewall.

Outer transparency implies that based on the messages alone, other parties cannot infer whether a firewall is used. Note that outer transparency requires the firewall to not change the functionality. However, this is not true for the other way around as a functionality-maintaining firewall could, for example, add a bit as a prefix to every message which would not break functionality but outer transparency. In [26], the firewalls are modelled to have an additional input

through which the content of its party's message can be changed. However, there are also firewalls without this input, for which a definition of *transparency* is given. Informally, such a firewall is transparent if it cannot be distinguished from a firewall that does not modify the communication and directly passes the message along. This indistinguishability is similar to our definition of outer transparency. However, strong transparency does not follow from their definition since, as discussed below, that would disallow the usage of feedback channels.

The authors of [46, 33] used transparency notions similar to ours to argue about the security of protocols in the case of a corrupted firewall: The *security against the firewall* [46] should follow directly from the security of the underlying protocol by considering the power of a corrupted firewall to be the same as the power of a maliciously corrupted communication partner. In our terminology, their idea is the following: Suppose that Π is secure against malicious corruptions. Then, the security of $\mathcal{P}$ can be argued by treating RF together with $\mathcal{P}'$ as a malicious party. Due to the inner transparency, the protocol run for $\mathcal{P}$ is indistinguishable from a run without the firewall against a malicious party $\mathcal{P}'$ and the security of the protocol thus implies the security of $\mathcal{P}$ (even when RF is corrupted). Similarly, the security of $\mathcal{P}'$ can be argued by treating RF together with $\mathcal{P}$ as a malicious party. Due to the outer transparency, the protocol run for $\mathcal{P}'$ is indistinguishable from a run without the firewall against a malicious party $\mathcal{P}$ and the security of the protocol thus implies the security of $\mathcal{P}'$ (even when RF is corrupted).

While the argument above neglects the implications of an impersonation attack, this is still the general idea we keep in mind. In our model that is using a shell and a body for each party, it is the task of each shell to handle the respective in- and outputs to both the body and the respective recipient, whether that is another shell of the same party or a shell of another party, e.g. a firewall or (in the case of no firewall) the communication partner. Hence, the body of a party does not need to be aware of the existence of a firewall and, therefore, does not need to change its content. It is now easy to see why inner transparency, corresponding to the fact that the body behaves identically whether there exists a firewall or not, is achievable in our setting. Consequently, this approach allows us to make similar arguments to guarantee security of the party as [46, 33].

However, in the model of Chakraborty et al., there is a need for a communication feedback channel between the firewall and its party. In the above commitment example, this feedback channel is used to inform the party about the blinding factor r that was used. As the party thus needs to be aware of the existence of the firewall, firewalls cannot provide inner transparency in the model of Chakraborty et al., and thus also not guarantee strong transparency. In this case, the argument does not hold and this feedback channel gives a corrupted firewall another possibility of sending corrupted messages to its party. Whether this additional attack vector allows an adversary to extract sensitive information or not depends on the specific handling of this, a statement as general as when not using a feedback channel is not possible.

A.2 Corrupted Firewalls

A firewall RF is modelled as a sub-party of main party $\mathcal{P}$ in the real world as visualised in Fig. 2. Every main party might have one (or more) firewalls but is not required to have one. Firewalls communicate via an immediate channel with the respective main parties and, for the communication onward to the network channel, pass on the message after modifying part of the content of the message without modifying the header.

For example, in the case of an authenticated channel, Alice would send $(\text{send}, \text{sid}, \text{Alice}, \text{Bob}, \text{msg})$ via the immediate channel to its firewall, which adapts the message to msg' and sends $(\text{send}, \text{sid}, \text{Alice}, \text{Bob}, \text{msg}')$ to the ideal authenticated channel functionality $\mathcal{F}_{\text{AUTH}}$, where msg' is a version of the message m where the possible covert channel is removed (often via rerandomisation).

This gives a good model of how messages are routed in a real-life network that passes messages/packets along a sequence of routers. Here, each device in a network only needs to know the address of its own router. Remark that RF sends messages in the name of $\mathcal{P}$ which could be used for an impersonation attack by a malicious RF , which is discussed further in Section 8. Apart from the interfaces for the input from its main party and the network channel, no further inputs from the outside are modelled/accepted, i.e., the environment $\mathcal{Z}$ cannot give further input to the firewall.

As a RF opens up a new attack vector to the setting, as they might also get corrupted, we provide corruption interfaces. Corruption handling and interfaces are part of the model handling rather than of the communication protocol. Further details are given in the attack Section 3.2. The environment can only make use of the corruption interfaces of RF . Note that RF does not provide a secret interface, as RF should and would not know a secret.

This model additionally differs from the one presented in [26] in that, there, an ideal functionality $\mathcal{F}$ has an explicit interface for the firewall RF , as $\mathcal{F}$ will be adapted to be “sanitisable”. This means that $\mathcal{F}$ has a set of interfaces to handle the in- and output from a party and a set of interfaces to handle the changes made by RF . Consequently, a party and its firewall cannot directly talk to each other as they have to communicate through the ideal functionality resulting in the requirement that all parties must use a firewall. On the modelling side, this change requires that every $\mathcal{F}$ must be adapted if it is used in a hybrid model; the authors of [26] propose a wrapper for their functionalities. Here, one can directly see that $\mathcal{F}$ learns the secret of the party as it has transferred the plain message from party to firewall.

The benefit of firewalls is using them to protect already existing protocols against subversion attacks. Hence, it is intuitive to only consider functionality-maintaining firewalls as these do not change the underlying functionality. As already discussed above, in [26], the firewalls are allowed to modify the main party’s inputs. This heavily impacts the definition of a functionality. Consider a commitment scheme as an example. Employing such a firewall that might change the committed element would lead to the commitment of a random value. While this is sufficient for functionalities using random inputs, more general

functionalities might require that a specific value is committed. One could argue that in this case, a firewall could simply be provided with inputs that do not change the value. However, this would give the firewalls—more specifically the inputs of the firewalls—the power to decide on the obtainable functionality. Hence, one could end up with two protocols for the *exact same* functionality, but with very different correctness guarantees depending on firewalls used.

Note that there is also an ambivalence present regarding informing the party of the value that was actually committed to. As can be seen in [26], a protocol implementing such an ideal functionality can be designed in such a way that no update is passed back to the party. While this means that no feedback channel is needed, the party can also no longer use the committed value later on in the protocol, as it is unknown to the party. If, instead, an update should always be returned to the party to allow this kind of functionality, a feedback channel is necessary. Since this channel is not needed in the underlying protocol, this requires a corresponding modification. Furthermore, using multiple firewalls with a feedback channel together requires additional management: All firewalls have to forward all received randomnesses to the party which in the end has to sum all of them up. Alternatively, the firewalls have to be adapted to wait for their outer firewall to answer with a randomness before updating their own randomness accordingly and passing it to next firewall such that the party will receive only one updated randomness.